

Министерство науки и высшего образования РФ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Ярославский государственный технический университет»
Кафедра «Информационные системы и технологии»

Курсовой проект защищен
с оценкой _____
Преподаватель
_____ Ю. Н. Шулева
«__» _____ 2022

**РАЗРАБОТКА ИНФОРМАЦИОННОЙ СИСТЕМЫ
«БЮРО ПО ТРУДОУСТРОЙСТВУ»**

Пояснительная записка к курсовому проекту
по дисциплине «Системы управления базами данных»

ЯГТУ 09.03.02-009 КП

Нормоконтролер
Преподаватель
_____ Ю. Н. Шулева
«__» _____ 2022

Проект выполнил
студент группы ЭИС-36
Д. Р. Вялитова
«__» _____ 2022

Министерство науки и высшего образования РФ
Федеральное государственное бюджетное образовательное учреждение высшего
образования
«Ярославский государственный технический университет»

Кафедра «Информационные системы и технологии»

ЗАДАНИЕ № 9
по курсовому проектированию

Студенту Вялитовой Диане Равильевне
институт «Цифровых систем» курс 3 группа ЭИС-36

I. Тема проекта и исходные данные

Разработка информационной системы по теме «Разработка информационной системы «Бюро по трудоустройству»

СУБД: MS SQL Server

Среда разработки клиентского приложения: Netbeans 8.2

II. Представить следующие материалы:

1) текстовые

Пояснительная записка к курсовому проекту, включающая: системный анализ предметной области, концептуальную схему, логическую структуру базы данных, описание приложения, руководство пользователя и техническое задание

2) Графические

Иллюстративные материалы представляются студентом на защиту в виде компьютерной презентации в формате MS Power Point

III. Рекомендуемая литература и материал:

1. Ткачев О.А. Создание и манипулирование базами данных средствами СУБД Microsoft SQL Server 2008 [Электронный ресурс]: Учебное пособие/ Ткачев О.А. – Электрон. текстовые данные. – М.: Московский городской педагогический университет, 2013. – 152 с. – Режим доступа: <http://www.bibliocomlectator.ru/book/?id=26613>

2. Чубукова И.А. Data Mining [Электронный ресурс]/ Чубукова И.А. – Электрон. текстовые данные. – М.: Интернет-Университет Информационных Технологий (ИНТУИТ), 2016. – 470 с. – Режим доступа: <http://www.bibliocomlectator.ru/book/?id=56315>.

IV. Дата выдачи задания _____

V. Срок сдачи законченного проекта _____

VI. Отметка о явке на консультацию:

1) _____ 2) _____ 3) _____
4) _____ 5) _____ 6) _____

Руководитель проекта _____ **Ю. Н. Шулева** зав. кафедрой _____ **С. Ю. Бойков**

Задание принял к исполнению « _____ » _____ Студент _____

РЕФЕРАТ

84 с., 46 рис., 14 табл., 5 источников, 5 прил.

MS SQL SERVER, СУБД, Java, Netbeans 8.2, Web-приложение

Задание:

Разработать информационную систему для компании, занимающейся трудоустройством. В системе хранятся данные о соискателях, работодателях, работниках бюро. Резюме могут разместить «Соискатели», пройдя регистрацию, вакансии могут разместить «Работодатели». На основании подходящих под вакансии резюме, и, наоборот, «Работник бюро» заключает сделку.

Разработать: работа с данными «Соискателей», работа с данными «Работодателей», работа с данными «Работников бюро», работа с заключением сделки.

Объектом исследования является автоматизированная информационная система для бюро по трудоустройству.

Актуальность разработки. На данный момент существует не так много сервисов для поиска работы, во время цифровизации, люди ищут вакансии «по знакомству», либо в непроверенных сервисах. Автоматизированная информационная система для компании, занимающейся трудоустройством, сможет помочь людям удовлетворить свои желания в поиске работы с помощью web-приложения с простым и понятным интерфейсом.

Цель работы:

- а) Автоматизация поиска трудоустройства;
- б) Хранение информации о вакансиях, резюме, данных пользователей системы;
- в) Удобство работы с информационной системой.

Методы исследования и используемые средства. Для реализации нашей информационной системы будет происходить обработка информации о входных данных «Соискателей» и «Работодателей». С помощью «Работника бюро» будет заключаться сделка.

Полученные результаты. Размещения резюме «Соискателя» и вакансии «Работодателя», на основе этого «Работник бюро» будет заключать сделку.

Эффективность. Восполнение желания пользователей о поиске работы и работников для своей компании.

Область применения. Реализация информационной системы в бюро трудоустройства.

Основные эксплуатационные характеристики. Обеспечивается надежность и работоспособность объекта, условия, которые ведут к его результативности.

В результате всего процесса происходит автоматизация информационной системы для заключения сделок, связанных с трудоустройством и поиском работников с помощью «Работника бюро».

Содержание

Введение	5
1.1. Системный анализ предметной области.....	6
1.2. Назначение проектируемой системы.....	9
1.3. Цели создания проектируемой системы.....	10
1.4. Задачи, решаемые с помощью проектируемой системы	10
2. Проектная часть	10
2.1. Функциональная модель	10
2.2. UML-проектирование.....	14
2.2.1. Диаграмма вариантов использования	14
2.2.2. Диаграмма классов.....	15
2.2.3. Диаграмма развертывания	15
2.2.4. Диаграмма компонентов	16
2.2.5. Диаграмма состояния	16
2.3. Проектирование базы данных	17
2.3.1. Концептуальное проектирование.....	17
2.3.2. Логическое проектирование	17
2.3.3. Физическое проектирование.....	24
2.3.4. Роли и права доступа к данным.....	36
2.4. Проектирование интерфейса клиентской части	37
3. Технологическая часть	47
3.1. Выбор программных средств.....	47
3.1.1. Выбор СУБД.....	47
3.1.2. Выбор средств разработки	48
3.2. Описание работы с программой.....	48
Заключение.....	50
Список использованных источников и литературы	51
Приложение А.....	52
Приложение Б	62
Приложение В.....	68
Приложение Г	69

Введение

Сегодня информационные системы — это неотъемлемая часть нашего мира, которые позволяют обрабатывать и хранить множество информации в понятном виде для любого пользователя. Особую потребность в системах такого рода испытывают предприятия, поскольку систематизированный сбор и обработка данных, а также быстрый доступ к ним, является приоритетом каждой уважающей себя компании.

Ежедневно в мире циркулирует огромный поток информации, который необходимо зафиксировать, предоставить или обработать, таким образом, чтобы информация не была искажена третьими лицами. Информационные системы, которые ориентированы под определенную предметную область, помогают нам зафиксировать основные данные той или иной части мира, таким образом, что искажение данных сводится к нулю. Все данные помещаются на хранение в память компьютера, что освобождает от лишней бумажной волокиты, а это предполагает, что все данные доступны лишь щелчком мыши.

Таким образом, актуальность системы для бюро по трудоустройству невозможно не оценить. Множество данных о работодателях и соискателях, а также резюме и вакансиях, которые хранятся в системе, а также позволяют совершить быстрый поиск. Доступ к такой информационной системе обеспечивает контроль данных всех обращений в данную контору.

1. Аналитическая часть

1.1. Системный анализ предметной области

База данных создается с целью отслеживания работы бюро по трудоустройству.

База данных должна содержать:

- а) Данные о должности работника
- б) Данные о адресе
- в) Данные о паспорте
- г) Данные о роли
- д) Данные о работнике
- е) Данные о пользователе
- ж) Данные о сделке
- з) Данные о работодателе
- и) Данные о соискателе
- к) Данные о вакансии
- л) Данные об образовании
- м) Данные о виде деятельности

В соответствии с предметной областью система строится с учётом следующих особенностей:

- а) «Соискатель», пройдя регистрацию размещает своё резюме;
- б) «Работодатель» может разместить несколько вакансий;
- в) Данные пользователей просматривает пользователи «Администратор» и «Работник бюро»;
- г) «Работник бюро» может заключить сделку;
- д) Администратор может заключить сделку, добавить работника, изменить или удалить вакансию или резюме.

Выделим базовые сущности этой предметной области:

1) Должность работника. Атрибуты:

- 1.1) id_должности работника
- 1.2) наименование должности работника

2) Адрес. Атрибуты:

- 2.1) id _адреса
- 2.2) Город
- 2.3) Улица
- 2.4) Дом

- 2.5) Квартира
- 3) Паспорт. Атрибуты:
 - 3.1) id_паспорта
 - 3.2) Серия
 - 3.3) Номер
 - 3.4) Дата выдачи
 - 3.5) Место выдачи
- 4) Роль. Атрибуты:
 - 4.1) id_роли
 - 4.2) Название роли
- 5) Пользователь. Атрибуты:
 - 5.1) id _пользователя
 - 5.2) Логин
 - 5.3) Пароль
 - 5.4) id _роли
- 6) Работник. Атрибуты:
 - 6.1) id _работника
 - 6.2) Фамилия
 - 6.3) Имя
 - 6.4) Отчество
 - 6.5) id _должности работника
 - 6.6) id _адреса
 - 6.7) id _паспорта
 - 6.8) id _пользователя
 - 6.9) Телефон
 - 6.10) Фото
 - 6.11) Эл. Почта
- 7) Работодатель. Атрибуты:
 - 7.1) id _работодателя
 - 7.2) Наименование компании
 - 7.3) id _адреса
 - 7.4) Телефон компании
 - 7.5) Почта компании
 - 7.6) Информация о компании
 - 7.7) Логотип компании
 - 7.8) id _пользователя
 - 7.9) Счет компании
- 8) Образование. Атрибуты:
 - 8.1) id _образования

- 8.2) Уровень образования
- 8.3) Наименование образования
- 9) Вид деятельности. Атрибуты:
 - 9.1) id_вида_деятельности
 - 9.2) Название вида деятельности

- 10) Соискатель. Атрибуты:
 - 10.1) id_соискателя
 - 10.2) Фамилия
 - 10.3) Имя
 - 10.4) Отчество
 - 10.5) Телефон
 - 10.6) Почта
 - 10.7) Информация
 - 10.8) Фото
 - 10.9) id_вида_деятельности
 - 10.10) Желаемая з/п
 - 10.11) id_образования
 - 10.12) id_пользователя

- 11) Вакансия. Атрибуты:
 - 11.1) id_вакансии
 - 11.2) Название вакансии
 - 11.3) id_вида_деятельности
 - 11.4) Уровень З/П
 - 11.5) id_образования
 - 11.6) id_работодателя

- 12) Сделка. Атрибуты:
 - 12.1) id_сделки
 - 12.2) id_работника
 - 12.3) id_работодателя
 - 12.4) Дата сделки
 - 12.5) Сумма сделки
 - 12.6) id_вакансии

Функции:

- 1) Функции пользователя «Соискатель»:
 - 1.1) Размещение резюме (автоматически при регистрации)
 - 1.2) Просмотр резюме
 - 1.3) Просмотр вакансий
- 2) Функции пользователя «Работодатель»
 - 2.1) Размещение вакансий

- 2.2) Просмотр резюме
- 2.3) Просмотр вакансий
- 3) Функции пользователя «Работник бюро»
 - 3.1) Просмотр резюме
 - 3.2) Просмотр вакансий
 - 3.3) Заключение сделки
- 4) Функции пользователя «Администратора»
 - 4.1) Просмотр резюме
 - 4.2) Просмотр вакансий
 - 4.3) Заключение сделки
 - 4.4) Добавление пользователя «Работник бюро»

Выводы. Система должна обеспечивать:

- а) Работу с данными пользователя «Соискатель»;
- б) Работу с данными пользователя «Работодатель»;
- в) Работу с данными пользователя «Работник бюро»
- г) Работу с вакансиями;
- д) Работу с резюме;
- е) Работу с заключением сделки;
- ж) Работу с оформлением документа сделки;

Бизнес-правила:

Бюро по трудоустройству зарегистрировано в Российской Федерации, пользователями программы могут быть только граждане и компании Российской Федерации.

В бюро по трудоустройству «Соискателю» и «Работодателю» не нужно искать друг друга, необходимо лишь разместить резюме и вакансию. «Работник бюро» по данным «Соискателя» и «Работодателя» сам подберет необходимое и составит сделку.

При регистрации «Соискателя» его профиль автоматически становится резюме.

1.2. Назначение проектируемой системы

Проектируемая система предназначена для автоматизации и эффективной результативности компании, занимающейся трудоустройством.

Она обеспечивает сбор, хранение и обработку данных, на основе которых обеспечивает вывод более точной информации для улучшения работоспособности пользователя данной системы.

Техническое задание по проектируемой информационной системе представлено в приложении А.

1.3. Цели создания проектируемой системы

Цели создания системы является обработка определенных данных, с помощью которых происходит наглядная реализация. На основании данного процесса будет происходить процесс трудоустройства соискателей с последующим заключением сделки.

1.4. Задачи, решаемые с помощью проектируемой системы

С помощью проектируемой системы осуществляются следующие задачи:

- 1) Хранение и обработка данных пользователей системы;
- 2) Реализация работы с данными пользователя «Соискатель»;
- 3) Реализация работы с данными пользователя «Работодатель»;
- 4) Реализация работы с данными пользователя «Работник бюро»;
- 5) Оформление сделок «Работником бюро»

2. Проектная часть

2.1. Функциональная модель



Рисунок 1 – Диаграмма IDEEO

Таблица 1 - Характеристики объектов модели IDEEF0

Объект	Название	Определение	Описание
Функция А0	Бюро по трудоустройству	Целью бюро по трудоустройству является автоматизация трудоустройства	Бюро готово искать работников для различных работодателей и вакансии для ищущих работу специалистов различного профиля
Вход А0	1) Обращение соискателя 2) Обращение работодателя	Обращение соискателя и работодателя, представленное в удобной для них форме	Обращение соискателя с помощью размещения резюме, обращение работодателя с помощью размещения вакансии.
Выход А0	Заклученная сделка	Результат обработки обращения соискателя и работодателя.	Документом, подтверждающим результат обработки, является документ сделки.
Контроль А0	1) Законодательство РФ, нормативные документы. 2) Правила пользования БД 3) Правила заполнения данных.	1)Законодательство РФ, нормативные документы, регулирующие работу данной организации. 2)Правила пользования БД – правила работы с данными, которые есть в бюро. 3)Правила заполнения данных - правила работы с	1)Законодательство РФ, нормативные документы, регулирующие работу данной организации. 2)Правила пользования БД – правила работы с данными, которые есть в бюро. 3)Правила заполнения данных - правила работы с

		данными, которые есть в бюро.	данными, которые есть в бюро.
Механизм A0	1) Работники бюро 2) Администратор	Сотрудники, прямо или косвенно участвующие в обработке обращения соискателя и работодателя.	Количество и состав сотрудников зависит от количества клиентов и перечня рабочих мест.

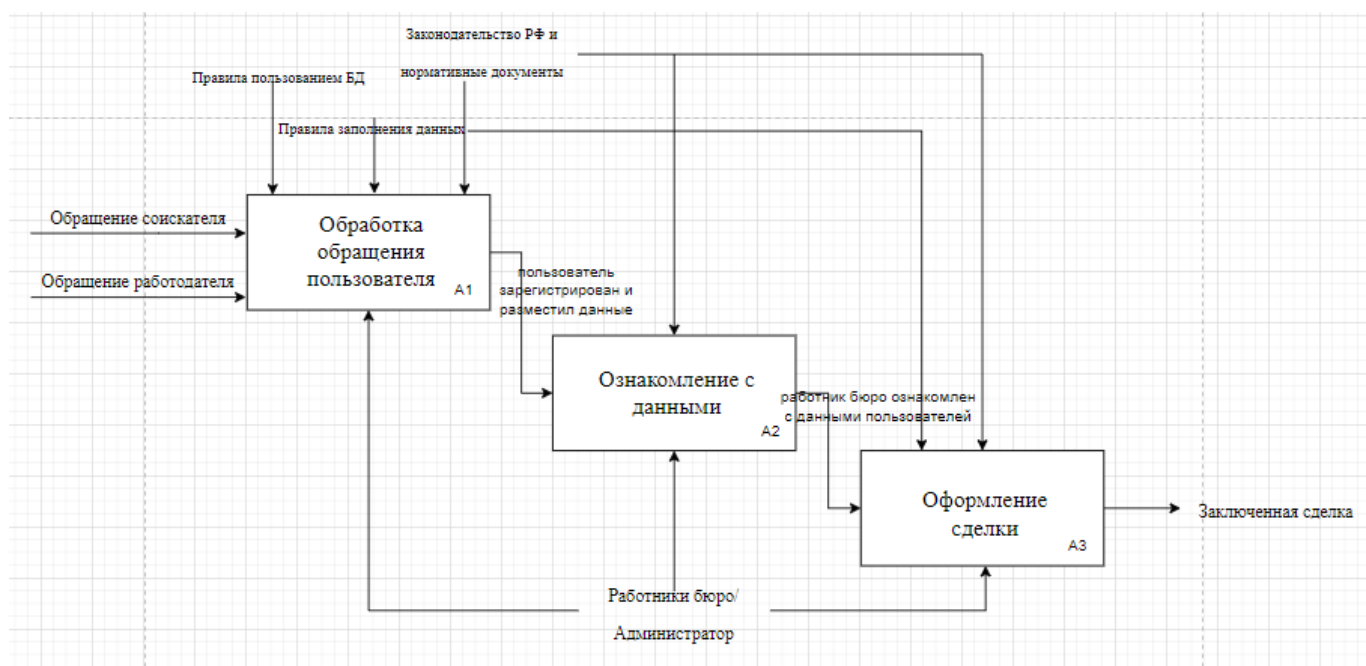


Рисунок 2 – Диаграмма IDEEF0 второго уровня

Таблица 2 - Характеристики объектов модели IDEEF0 второго уровня

Объект	Название	Определение	Описание
Функция A1	Обработка обращения пользователя	Приведение желаний пользователя в соответствие с возможностями бюро	Составление списка вакансий, списков резюме клиентов-соискателей
Функция A2	Ознакомление с данными	Анализ наличия соответствующих должностей с уточнением оплаты и поиск требуемых	Поиск работодателей и соискателей

		вакансий или специалистов	
Функция АЗ	Оформление сделки	Оформление сделки об обработке бюро обращений работодателей и соискателей	Подготовка отчетного документа, подтверждающего заключение сделки
Вход А1	1) Обращение соискателя 2) Обращение работодателя	Обращение соискателя и работодателя, представленное в удобной для них форме	Обращение соискателя с помощью размещения резюме, обращение работодателя с помощью размещения вакансии.
Выход АЗ	Заклученная сделка	Результат обработки обращения соискателя и работодателя.	Документом, подтверждающим результат обработки, является документ сделки.
Контроль А1	Правила пользования БД	Правила пользования БД – правила работы с данными, которые есть в бюро	Правила пользования БД – правила работы с данными, которые есть в бюро
Контроль А1 Контроль А2	Правила заполнения данных.	Правила заполнения данных - правила работы с данными, которые есть в бюро.	Правила заполнения данных - правила работы с данными, которые есть в бюро.
Контроль А1 Контроль А2 Контроль А3	Законодательство РФ, нормативные документы.	Законодательство РФ, нормативные документы, регулирующие работу данной организации.	Законодательство РФ, нормативные документы, регулирующие работу данной организации

Механизм А1 Механизм А2 Механизм А3	Работники бюро/администратор	Сотрудники, прямо или косвенно участвующие в обработке обращения соискателя и работодателя.	Количество и состав сотрудников зависит от количества клиентов и перечня рабочих мест.
---	---------------------------------	---	---

2.2. UML-проектирование

2.2.1. Диаграмма вариантов использования

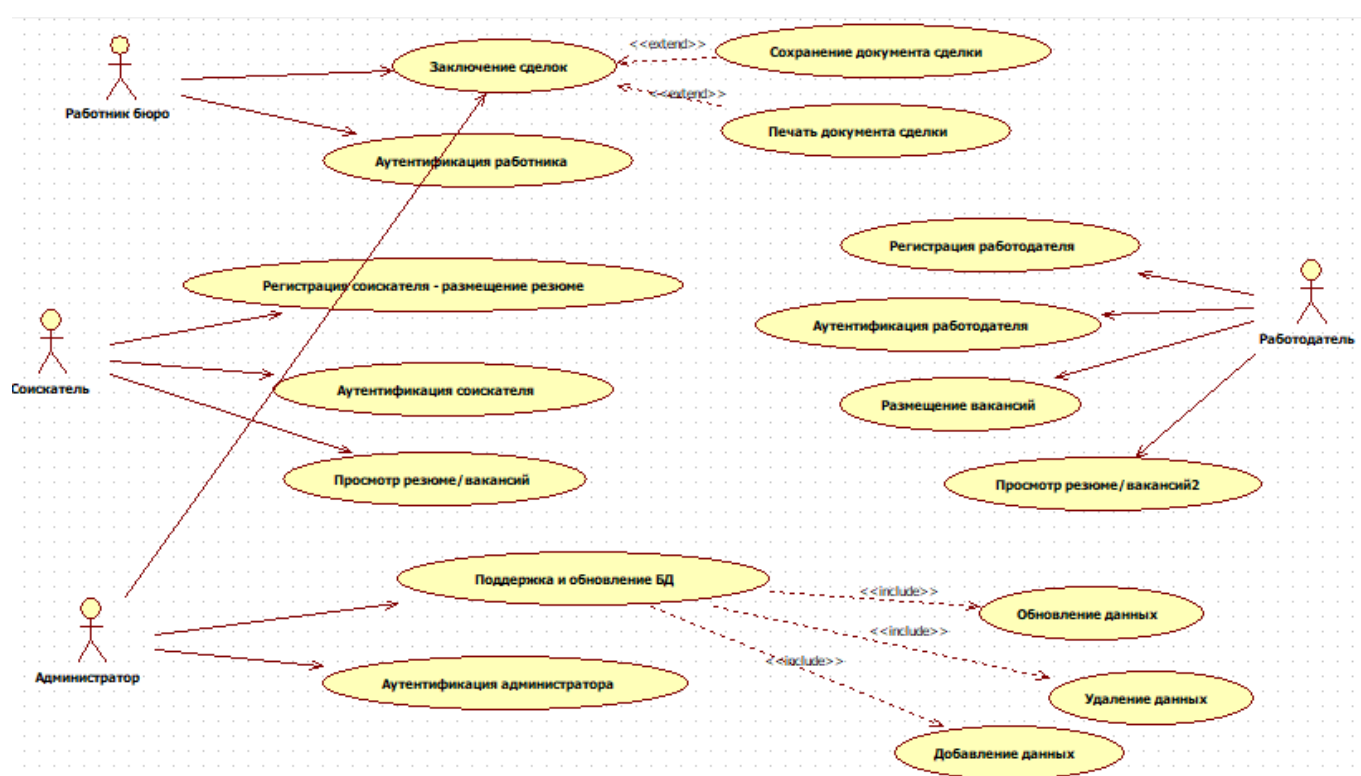


Рисунок 3 – Диаграмма вариантов использования

В данной диаграмме, которая указана на рисунке 4, присутствуют 4 актера. На диаграмме показаны основные функции актеров.

2.2.2. Диаграмма классов

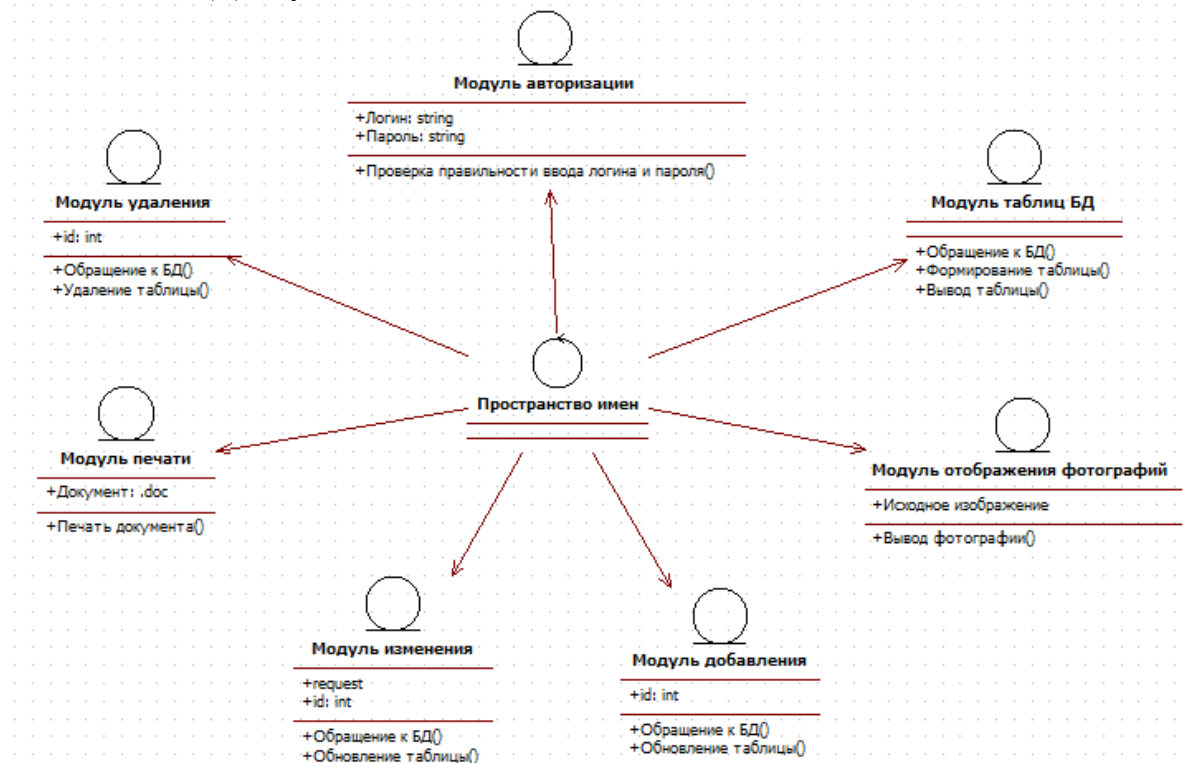


Рисунок 4 – Диаграмма классов

Данная диаграмма, указанная на рисунке 4, дает общее представление о приложении. Указывает компоненты (атрибут), которые являются вспомогательной частью функционирования того или иного модуля.

2.2.3. Диаграмма развертывания

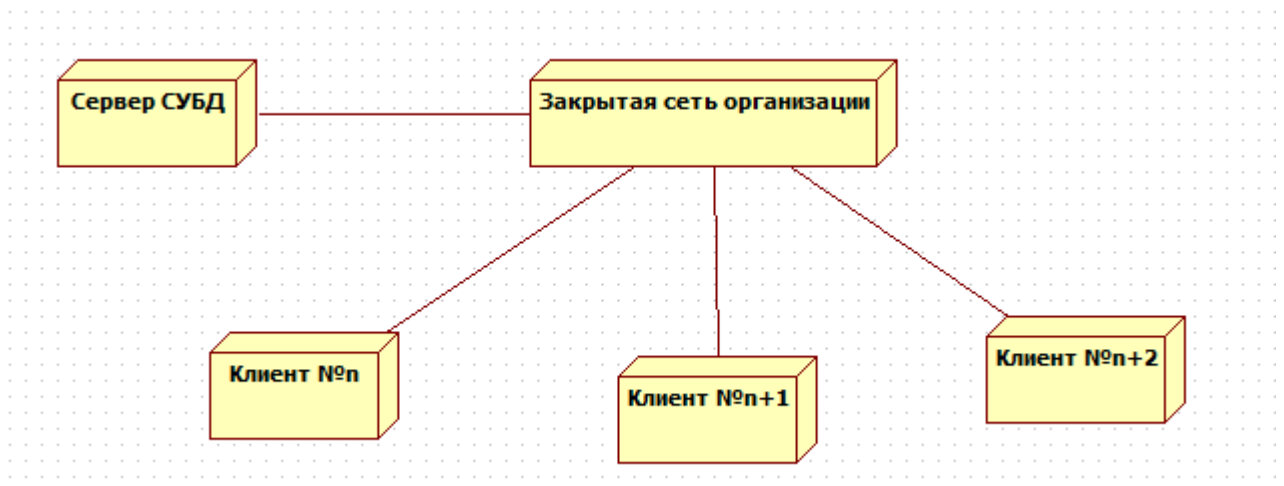


Рисунок 5 – Диаграмма развертывания

Диаграмма на рисунке 5, демонстрирует процесс взаимодействия клиентов с сервером СУБД через закрытую сеть организации.

2.2.4. Диаграмма компонентов

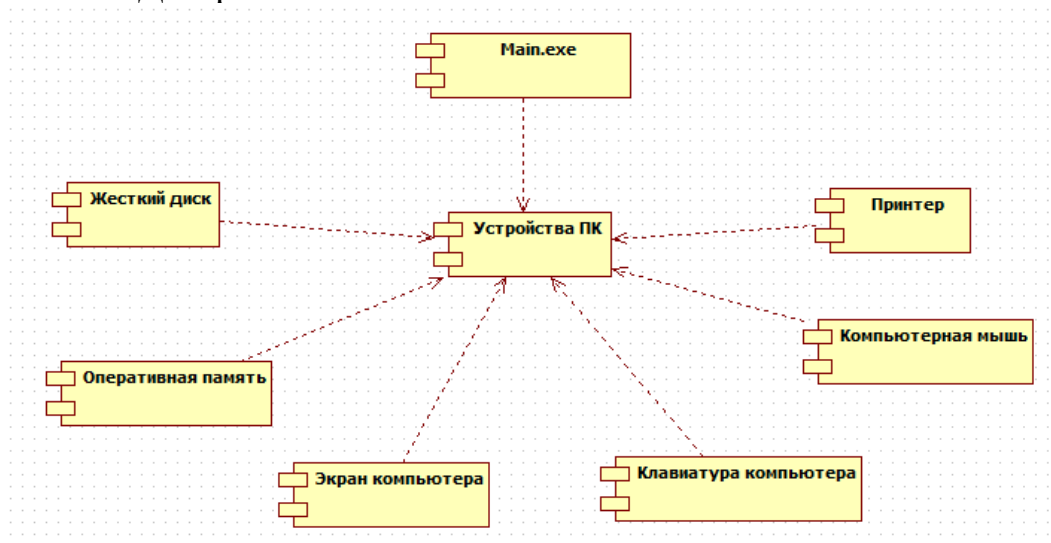


Рисунок 6 – Диаграмма компонентов

В данной диаграмме, указанной на рисунке 7, рассматриваются компоненты ПК, которые обеспечивают функционирование работы с компьютером.

2.2.5. Диаграмма состояния

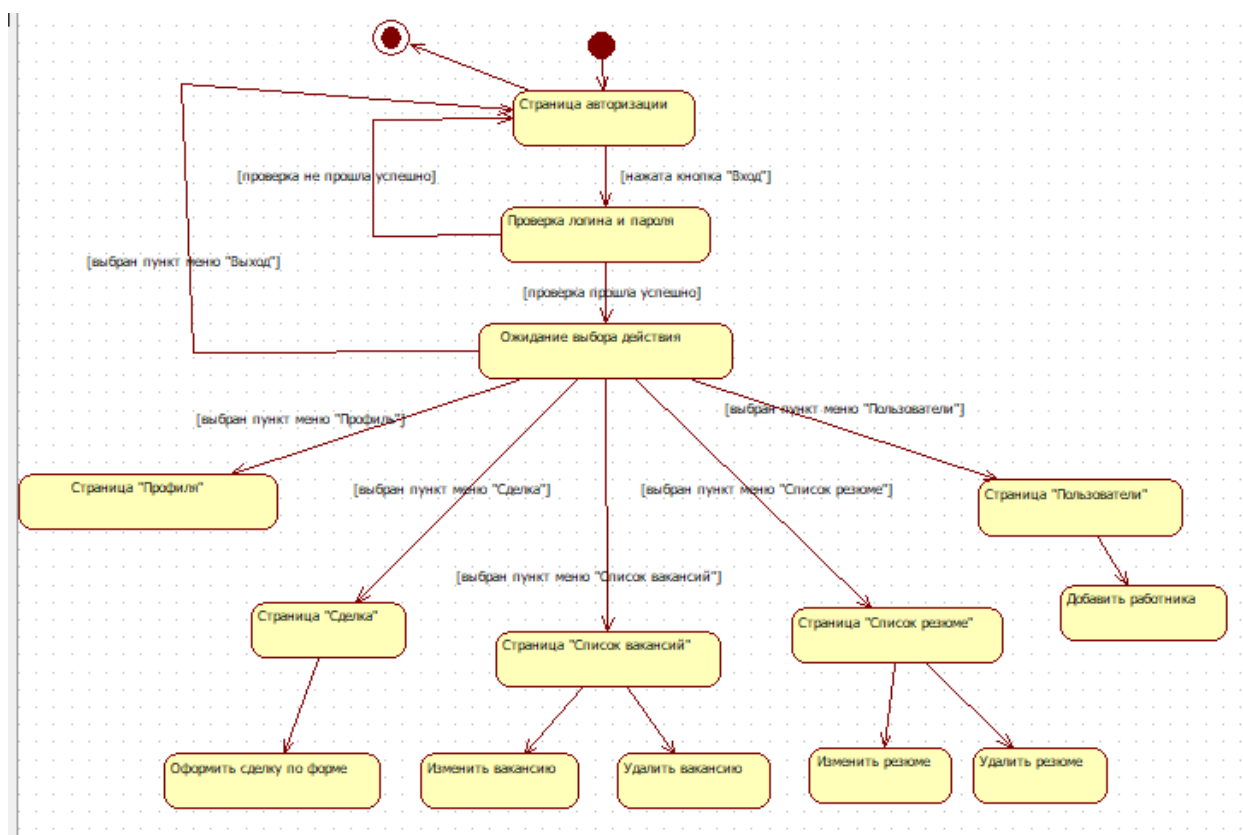


Рисунок 7 – Диаграмма состояния

Данная диаграмма на рисунке 7 расписывает процесс работы с информационной системой, состояниями, которые взаимодействуют друг с другом по определенному порядку (со стороны «Администратора»).

2.3. Проектирование базы данных

2.3.1. Концептуальное проектирование

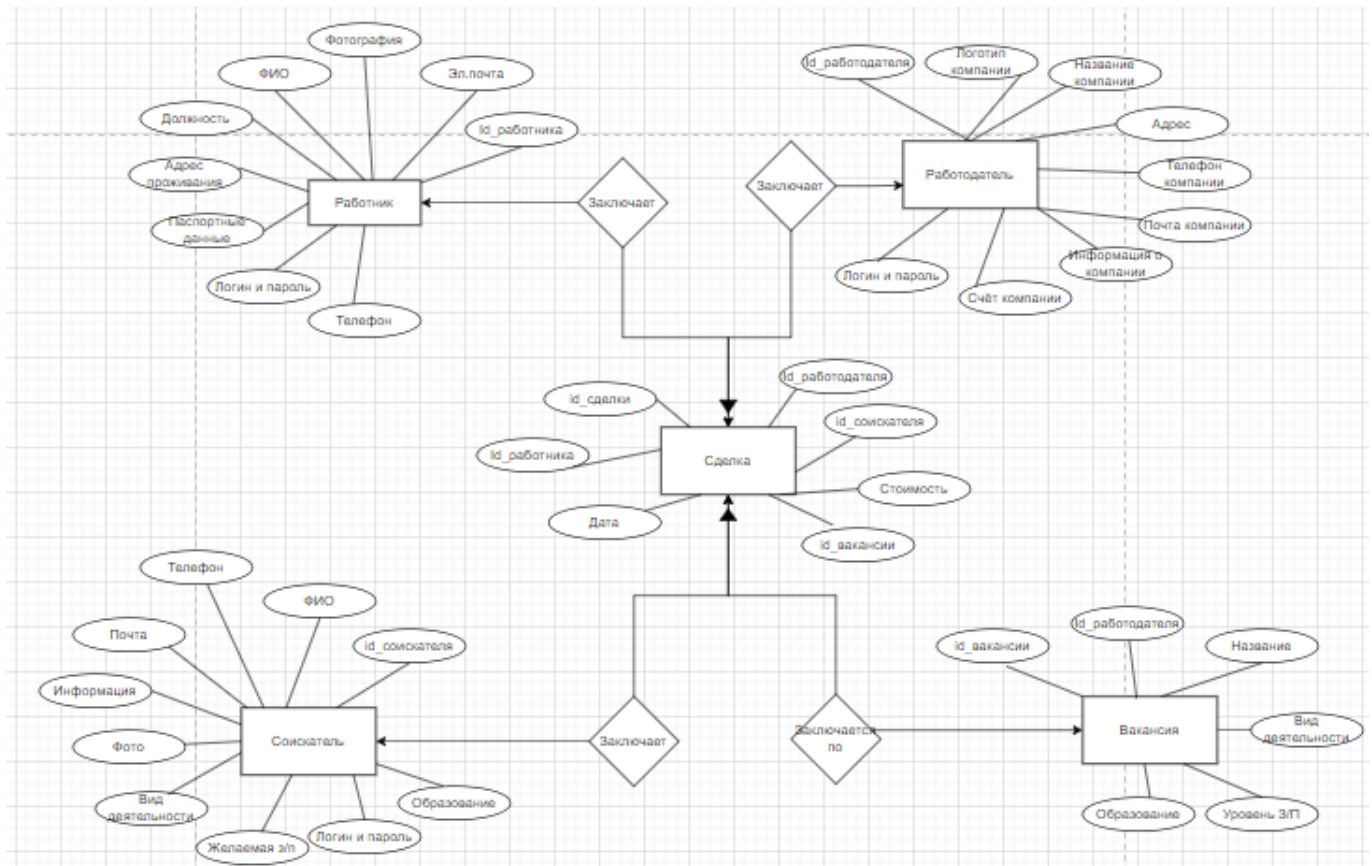


Рисунок 8 – Концептуальная модель

Все объекты, обозначающие вещи, изображаются в виде прямоугольника. Атрибуты, характеризующие объект - в виде овала, а связи между объектами - ромбами.

Например, на рисунке 8 указана связь работник и сделка, которые имеют одинаковый атрибут id_работника, через который обеспечивается связь между ними. Они имеют связь один-ко-многим.

Мощность связи:

- 1) Работник-Сделка: один ко многим;
- 2) Работодатель-Сделка: один ко многим;
- 3) Соискатель-Сделка: один ко многим;
- 4) Вакансия-Сделка: один ко многим.

2.3.2. Логическое проектирование

Логическое проектирование – это описание и построение схем связей между элементами данных безотносительно к их содержанию и среде хранения.

Цель логического проектирования:

- 1) выборка наиболее эффективной структуры данных,
- 2) обеспечения быстрого доступа к данным;
- 3) исключения дублирования данных, обеспечения целостности данных таким образом, чтобы при изменении одних объектов автоматически происходило соответствующее изменение связанных с ними объектов.

До приведения в нормальную форму.

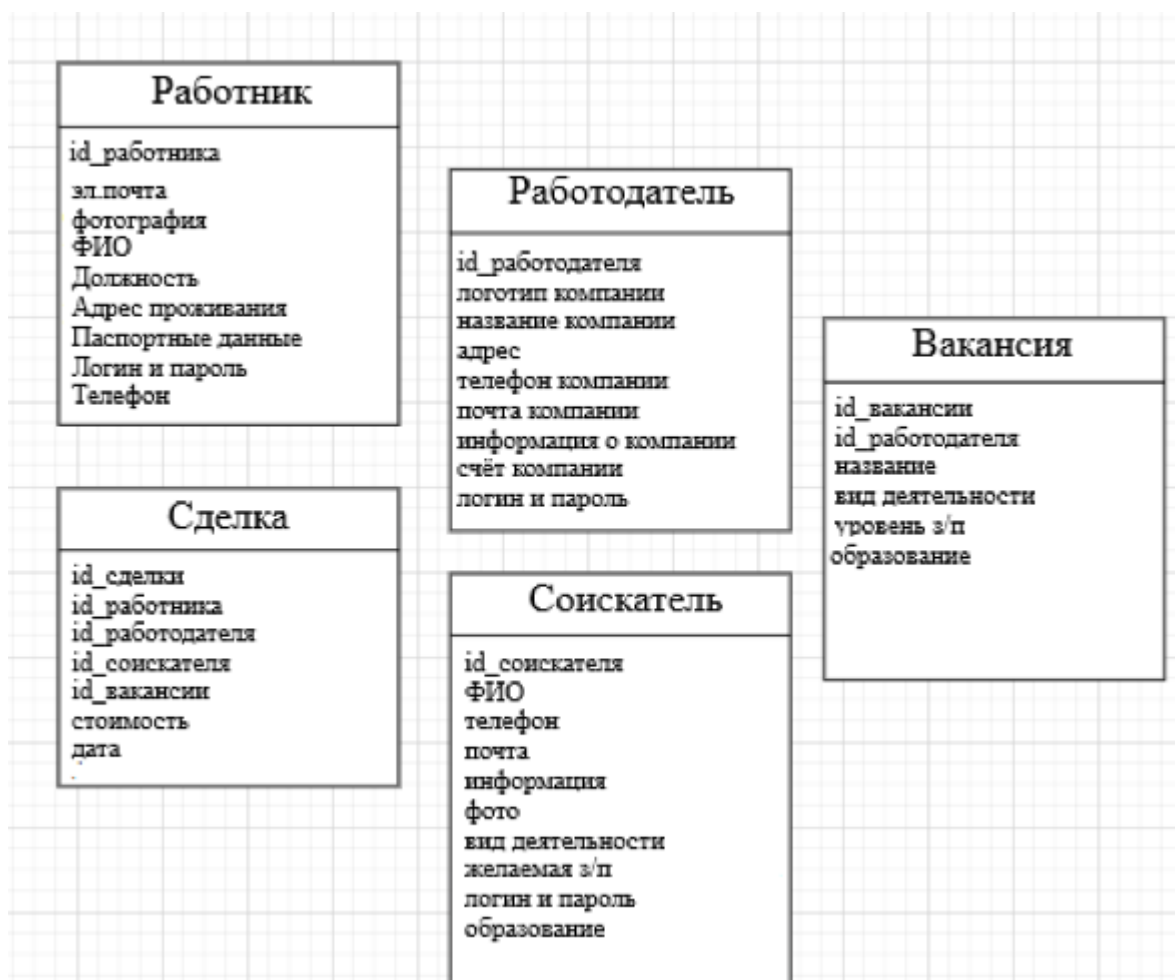


Рисунок 9 – Таблицы в неатомарном виде

Атрибуты на рисунке 9 не во всех таблицах удовлетворяют требованиям 1НФ.

1) Таблица «Работник»

Атрибуты «ФИО», «Паспортные данные», «Адрес проживания», «Логин и пароль».

В частности, поле «Адрес» не является атомарным, т.к. содержит информацию о стране, городе, улице, доме и квартире. Поэтому целесообразно выделить данную информацию в отдельных полях «город_работника», «улица_работника», «дом_работника», «квартира_работника».

Атрибут «Паспортные_данные» имеет в себе данные о номере и серии паспорта, также данные, когда выдан и кем. Заменяем этот атрибут на «серия_паспорта», «номер_паспорта», «дата_выдачи_паспорта», «место_выдачи_паспорт».

Атрибут ФИО является составным и его необходимо разделить на 3 атрибута - Фамилия, Имя, Отчество. Вносим в данную таблицу вместо «ФИО», атрибуты «фамилия_работника», «имя_работника», «отчество_работника».

Атрибут Логин и пароль является составным и его необходимо разделить на 2 атрибута – Логин, Пароль. Вносим в данную таблицу вместо «Логин и пароль», атрибуты «логин_работника», «пароль_работника».

2) Таблица «Работодатель»

Атрибуты «Адрес», «Логин и пароль».

Поле «Адрес» не является атомарным, т.к. содержит информацию о стране, городе, улице, доме и квартире. Поэтому целесообразно выделить данную информацию в отдельных полях «город_работодателя», «улица_работодателя», «дом_работодателя», «квартира_работодателя».

Атрибут Логин и пароль является составным и его необходимо разделить на 2 атрибута – Логин, Пароль. Вносим в данную таблицу вместо «Логин и пароль», атрибуты «логин_работодателя», «пароль_работодателя».

3) Таблица «Соискатель»

Атрибуты «ФИО», «Логин и пароль», «Образование».

Атрибут ФИО является составным и его необходимо разделить на 3 атрибута - Фамилия, Имя, Отчество. Вносим в данную таблицу вместо «ФИО», атрибуты «фамилия_соискателя», «имя_соискателя», «отчество_соискателя».

Атрибут Логин и пароль является составным и его необходимо разделить на 2 атрибута – Логин, Пароль. Вносим в данную таблицу вместо «Логин и пароль», атрибуты «логин_соискателя», «пароль_соискателя».

Атрибут Образование является составным и его необходимо разделить на 3 атрибута – Уровень образования, Направление образования, Скан документа. Вносим в таблицу вместо «Образование», атрибуты «уровень_образования_соискателя», «направление_образования_соискателя».

4) Таблица «Вакансия»

Атрибут «Образование».

Атрибут Образование является составным и его необходимо разделить на 3 атрибута – Уровень образования, Направление образования, Скан документа. Вносим в таблицу вместо «Образование», атрибуты «уровень_образования_вакансии», «направление_образования_вакансии».

5) Таблица «Сделка» не имеет атомарных столбцов.

Определим первичные ключи данных таблиц:

- 1) В таблице «Работник» первичным ключом будет атрибут «id_работника», который мы вводим в таблицу.
- 2) В таблице «Работодатель» первичным ключом будет атрибут «id_работодателя», который мы вводим в таблицу.
- 3) В таблице «Соискатель» первичным ключом будет атрибут «id_соискателя», который мы вводим в таблицу.
- 4) В таблице «Сделка» первичным ключом будет атрибут «id_сделки», который мы вводим в таблицу.
- 5) В таблице «Вакансия» первичным ключом будет атрибут «id_вакансии», который мы вводим в таблицу.

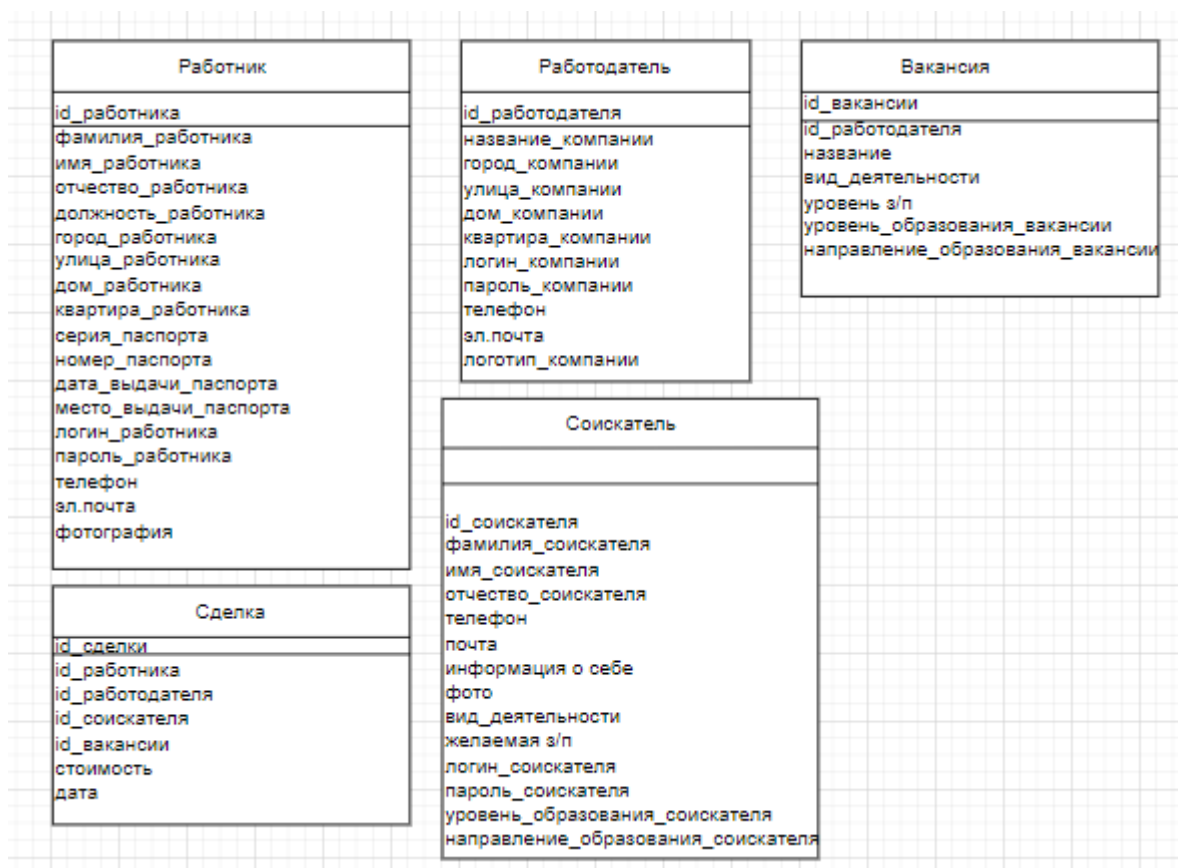


Рисунок 10 – Таблицы, приведенные в 1NF

Теперь схема на рисунке 10 отвечает требованиям 1NF.

Для приведения ко 2NF, нам надо сделать так, чтобы каждый неключевой атрибут функционально полно зависит от первичного ключа.

Вынесем атрибуты «серия_паспорта», «номер_паспорта», «дата_выдачи_паспорта», «место_выдачи_паспорта» из таблицы «Работник» в отдельную таблицу «Паспорт», заменив их на атрибут «id_паспорта».

Вынесем атрибуты «город_работника», «улица_работника», «дом_работника», «квартира_работника» из таблицы «Работник» в отдельную таблицу «Адрес», заменив их на атрибут «id_адреса».

Аналогично вынесем атрибуты «страна_работодателя», «город_работодателя», «улица_работодателя», «дом_работодателя», «квартира_работодателя» из таблицы «Соискатель» в таблицу «Адрес», заменив их на атрибут «id_адреса».

Вынесем атрибуты «логин_работника», «пароль_работника» из таблицы «Работник» в таблицу «Пользователь», заменив их на атрибут «id_пользователя».

Аналогично вынесем атрибуты «логин_работодателя», «пароль_работодателя» из таблицы «Работодатель» в таблицу «Пользователь», заменив их на атрибут «id_пользователя».

Аналогично вынесем атрибуты «логин_соискатель», «пароль_соискатель» из таблицы «Соискатель» в таблицу «Пользователь», заменив их на атрибут «id_пользователя».

Из таблицы «Работник» мы вынесем атрибут «должность» в отдельную таблицу
Должность работника (id_должности_работника,
наименование_должности_работника), заменив данный атрибут в таблице
«Работник» на «id_должности_работника».

Из таблицы «Соискатель» мы вынесем атрибут «вид деятельности» в
отдельную таблицу Вид деятельности (id_вида_деятельности,
наименование_вида_деятельности), заменив данный атрибут в таблице «Соискатель»
на «id_вида_деятельности».

Аналогично из таблицы «Вакансия» мы вынесем атрибут «вид деятельности» в
отдельную таблицу Вид деятельности (id_вида_деятельности,
наименование_вида_деятельности), заменив данный атрибут в таблице «Вакансия»
на «id_вида_деятельности».

Из таблицы «Соискатель» мы вынесем атрибут «образование» в отдельную
таблицу «Образование» (id_образование, уровень_образования,
направление_образования), заменив данный атрибут в таблице «Соискатель» на
«id_образования».

Аналогично из таблицы «Вакансия» мы вынесем атрибут «образование» в
отдельную таблицу «Образование» (id_образование, уровень_образования,

направление_образования), заменив данный атрибут в таблице «Вакансия» на «id_образования».

Сейчас укажем связи между первичными и внешними ключами:

1) Таблицы «Работник» и «Должность работника» связаны между собой полем «id_должность_работника» (в таблице «Работник» поле «id_должность_работника»- внешний ключ).

2) Таблицы «Работник» и «Адрес» связаны между собой полем «id_адреса» (в таблице «Работника» поле «id_адрес»- внешний ключ).

3) Таблицы «Работник» и «Паспорт» связаны между собой полем «id_паспорта» (в таблице «Работник» поле «id_паспорта»- внешний ключ).

4) Таблицы «Работник» и «Пользователь» связаны между собой полем «id_пользователя» (в таблице «Работник» поле «id_пользователя»- внешний ключ).

5) Таблицы «Работодатель» и «Адрес» связаны между собой полем «id_адреса» (в таблице «Работодатель» поле «id_адреса»- внешний ключ).

6) Таблицы «Работодатель» и «Пользователь» связаны между собой полем «id_пользователя» (в таблице «Работодатель» поле «id_пользователя»- внешний ключ).

7) Таблицы «Соискатель» и «Вид_деятельности» связаны между собой полем «id_вида_деятельности» (в таблице «Соискатель» поле «id_вида_деятельности»- внешний ключ).

8) Таблицы «Соискатель» и «Образование» связаны между собой полем «id_образование» (в таблице «Соискатель» поле «id_образование»- внешний ключ).

9) Таблицы «Соискатель» и «Пользователь» связаны между собой полем «id_пользователя» (в таблице «Соискатель» поле «id_пользователя»- внешний ключ).

10) Таблицы «Вакансия» и «Вид_деятельности» связаны между собой полем «id_вида_деятельности» (в таблице «Вакансия» поле «id_вида_деятельности»- внешний ключ).

11) Таблицы «Вакансия» и «Работодатель» связаны между собой полем «id_работодателя» (в таблице «Вакансия» поле «id_работодателя»- внешний ключ).

12) Таблицы «Вакансия» и «Образование» связаны между собой полем «id_образования» (в таблице «Вакансия» поле «id_образования»- внешний ключ).

13) Таблицы «Сделка» и «Работник» связаны между собой полем «id_работника» (в таблице «Сделка» поле «id_работника»- внешний ключ).

14) Таблицы «Сделка» и «Работодатель» связаны между собой полем «id_работодателя» (в таблице «Сделка» поле «id_работодателя»- внешний ключ).

15) Таблицы «Сделка» и «Соискатель» связаны между собой полем «Соискатель» (в таблице «Сделка» поле «id_соискателя»- внешний ключ).

16) Таблицы «Сделка» и «Вакансия» связаны между собой полем «id_вакансия» (в таблице «Сделка» поле «id_вакансия»- внешний ключ).

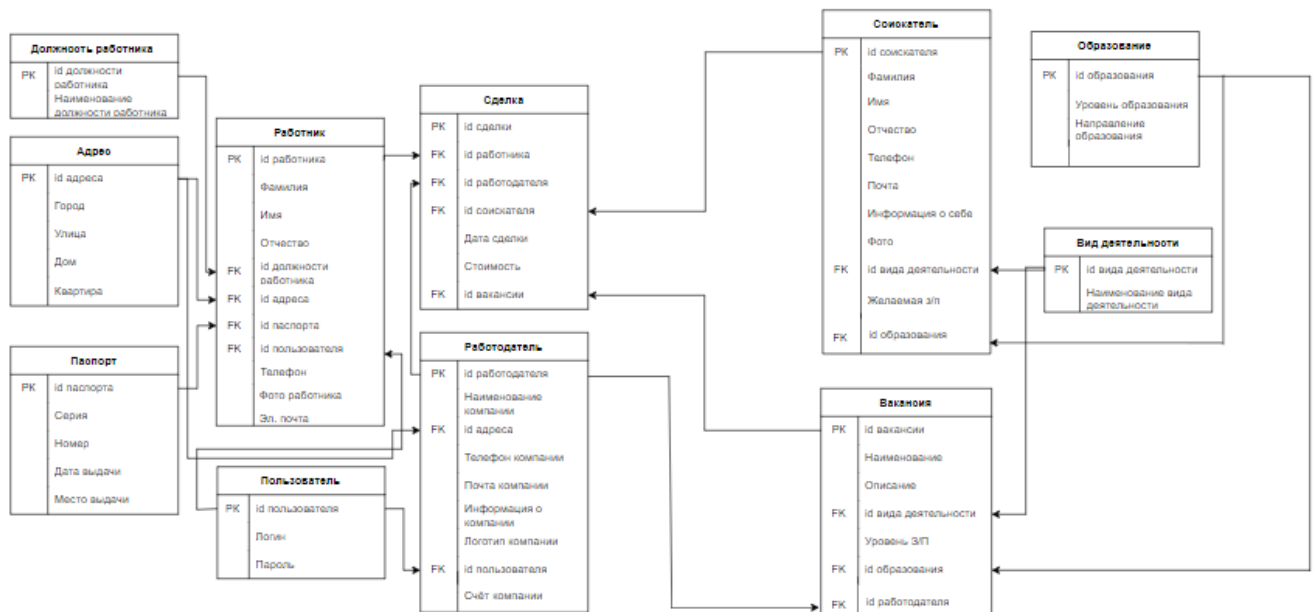


Рисунок 11 – Таблицы, приведенные в 2NF

Наша схема, изображенная на рисунке 11, приведена ко 2NF.

Для 3NF в таблицах не должно быть транзитивных зависимостей. Такой зависимости в таблицах нет.

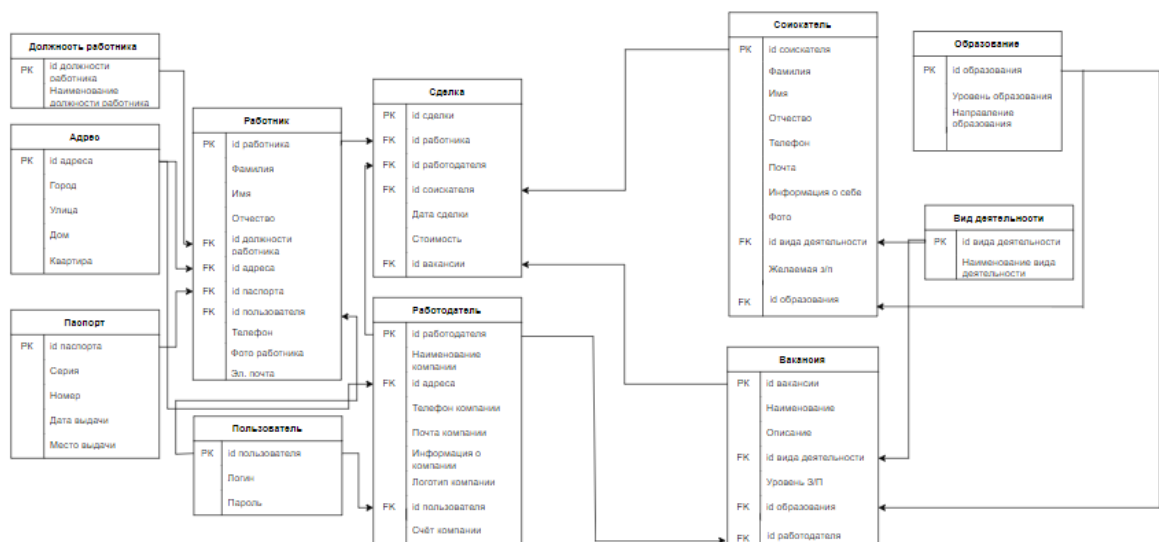


Рисунок 12 – Таблицы, приведенные в 3NF

Теперь БД на рисунке 13 является нормализованной.

Глядя на описание варианта и бизнес-правила, можно заметить, что:

1. Отсутствует информация о ролях пользователей системы.

Для этого надо создать новую таблицу «Роль» с атрибутами «id_роли» и «наименование_роли». Связать данную таблицу с таблицей «Пользователь».

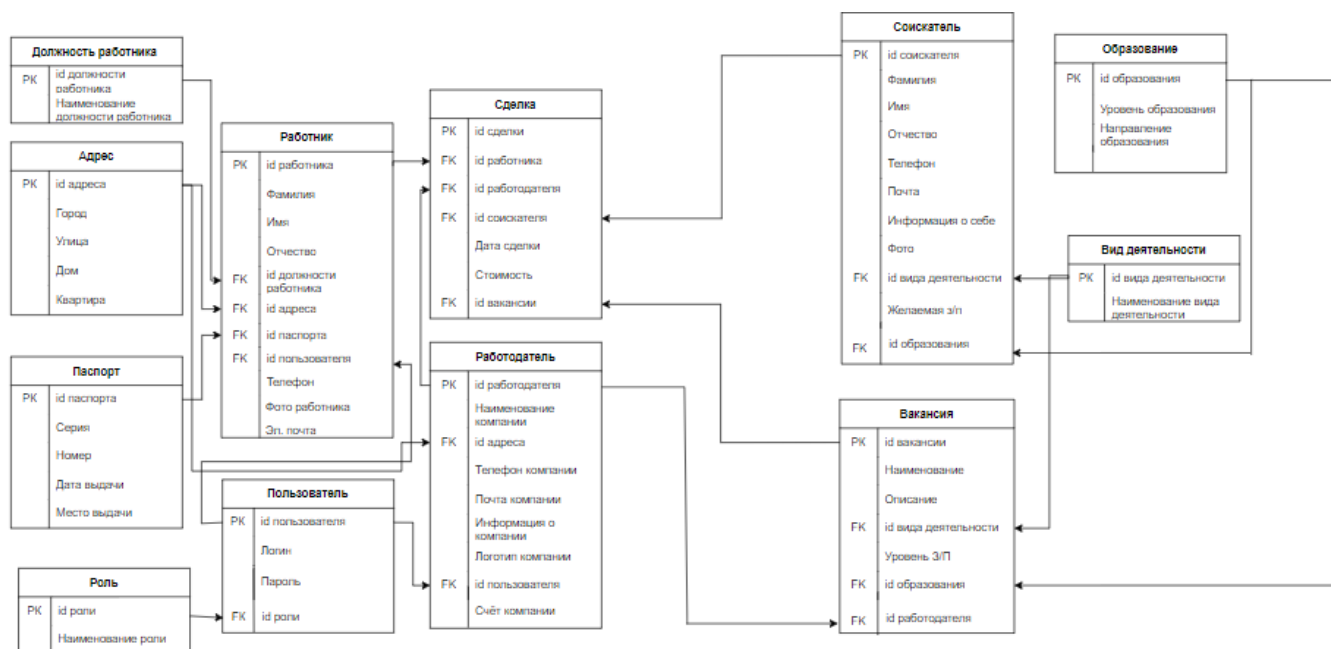


Рисунок 13 - Результат логического проектирования

Первичные ключи в рисунке 13:

- 1) В таблице «Должность работника» первичный ключ – поле «id_должности_работника»;
- 2) В таблице «Адрес» первичный ключ – поле «id_адреса»;
- 3) В таблице «Паспорт» первичный ключ – поле «id_паспорта»;
- 4) В таблице «Пользователь» первичный ключ – поле «id_пользователя»;
- 5) В таблице «Роль» первичный ключ – поле «id_роли»;
- 6) В таблице «Сделка» первичный ключ – поле «id_сделки»;
- 7) В таблице «Работодатель» первичный ключ – поле «id_работодателя»;
- 8) В таблице «Соискатель» первичный ключ – поле «id_соискателя»;
- 9) В таблице «Вакансия» первичный ключ – поле «id_вакансии»;
- 10) В таблице «Образование» первичный ключ – поле «id_образования»;
- 11) В таблице «Вид деятельности» первичный ключ – поле «id_вида_деятельности»;
- 12) В таблице «Работник» первичный ключ – поле «id_работника»;

2.3.3. Физическое проектирование

Сделать описание каждого столбца каждой таблицы с помощью указанного ниже способа

Таблица 3 – Таблица «Должность работника»

Имя поля	Тип данных	Ограничения	Описание
id_должности_работника	INT с атрибутом IDENTITY	PRIMARY KEY Автоматическое увеличение на 1.	Код должности работника, автовычисляемое поле
Наименование_должности_работника	varchar(50)	NOT NULL	Наименование должности работника

Описание таблицы 3:

Тип данных: Поле «id_должности работника» относится к типу данных – INT с атрибутом IDENTITY. Поле «Наименование_должности_работника» относится к символьному типу, где varchar представляет строку переменной длины, а в скобках указывается длина хранимой строки.

Ограничения: Поле «id_должности работника» имеет ограничения PRIMARY KEY - первичный ключ, автоматическое увеличение на 1. Поле «Наименование_должности_работника» должны быть обязательно заполненными (NOT NULL).

Таблица 4 – Таблица «Адрес»

Имя поля	Тип данных	Ограничения	Описание
id_адреса	INT с атрибутом IDENTITY	PRIMARY KEY Автоматическое увеличение на 1.	Код адреса, автовычисляемое поле
Город	varchar(50)	NOT NULL	Название города
Улица	varchar(50)	NULL	Название улицы
Дом	varchar(10)	NULL	Номер дома
Квартира	varchar(10)	NULL	Номер квартиры

Описание таблицы 4:

Тип данных: Поле «id_адреса» относится к типу данных – INT с атрибутом IDENTITY. Поля «Город», «Улица», «Дом», «Квартира» относятся к символьному типу, где varchar представляет строку переменной длины, а в скобках указывается длина хранимой строки.

Ограничения: Поле «id_адреса» имеет ограничения PRIMARY KEY - первичный ключ, автоматическое увеличение на 1. Поле «Город» должны быть обязательно заполненными (NOT NULL). Поля «Улица», «Дом», «Квартира» не обязательны к заполнению.

Таблица 5 – Таблица «Паспорт»

Имя поля	Тип данных	Ограничения	Описание
id_паспорта	INT с атрибутом IDENTITY	PRIMARY KEY Автоматическое увеличение на 1.	Код адреса, автовычисляемое поле
Серия	varchar(4)	NOT NULL Формат: CCCC	Серия паспорта
Номер	varchar(6)	NOT NULL Формат: НННННН	Номер паспорта
Дата_выдачи	Date	NOT NULL Формат: YYYY-MM-DD	Дата выдачи паспорта
Место выдачи	varchar(50)	NOT NULL	Место выдачи паспорта

Описание таблицы 5:

Тип данных: Поле «id_паспорта» относится к типу данных – INT с атрибутом IDENTITY. Поля «Серия», «Номер», «Место_выдачи» относятся к символьному типу, где varchar представляет строку переменной длины, а в скобках указывается длина хранимой строки. Поле «Дата_выдачи» относится к типу даты/время - date.

Ограничения: Поле «id_паспорта» имеет ограничения PRIMARY KEY - первичный ключ, автоматическое увеличение на 1. Поля «Серия», «Номер», «Место_выдачи» «Дата_выдачи» должны быть обязательно заполненными (NOT NULL).

Таблица 6 – Таблица «Роль»

Имя поля	Тип данных	Ограничения	Описание
id_роли	INT с атрибутом IDENTITY	PRIMARY KEY Автоматическое увеличение на 1.	Код адреса, автовычисляемое поле
Наименование_роли	varchar(50)	NOT NULL	Наименование роли

Описание таблицы 6:

Тип данных: Поле «id_роли» относится к типу данных – INT с атрибутом IDENTITY. Поле «Наименование_роли» относится к символьному типу, где varchar представляет строку переменной длины, а в скобках указывается длина хранимой строки.

Ограничения: Поле «id_роли» имеет ограничения PRIMARY KEY - первичный ключ, автоматическое увеличение на 1. Поле «Наименование_роли» должно быть обязательно заполненными (NOT NULL).

Таблица 7 – Таблица «Пользователь»

Имя поля	Тип данных	Ограничения	Описание
id_пользователя	INT с атрибутом IDENTITY	PRIMARY KEY Автоматическое увеличение на 1.	Код адреса, автовычисляемое поле
Логин	varchar(255)	NOT NULL Строка в нижнем регистре, состоящая из символов латиницы и цифр.	Логин для авторизации в приложении
Пароль	varchar(255)	NOT NULL	Пароль учётной записи для авторизации в приложении.

		Строка, состоящая из символов латиницы и цифр.	
id_роли	INT	FOREIGN KEY, ON DELETE CASCADE NOT NULL	ВК, связь: Роль (id_роли)

Описание таблицы 7:

Тип данных: Поле «id_пользователя» относится к типу данных – INT с атрибутом IDENTITY. Поле «id_роли» - int. Поля «Логин», «Пароль» относятся к символьному типу, где varchar представляет строку переменной длины, а в скобках указывается длина хранимой строки.

Ограничения: Поле «id_пользователя» имеет ограничения PRIMARY KEY - первичный ключ, автоматическое увеличение на 1. Поле «id_роли» имеет ограничения FOREIGN KEY- внешний ключ, ON DELETE CASCADE- удаляет дочерние строки при удалении родительского ключа. Поля «Логин», «Пароль» должны быть обязательно заполненными (NOT NULL).

Таблица 8 – Таблица «Работник»

Имя поля	Тип данных	Ограничения	Описание
id_работника	INT с атрибутом IDENTITY	PRIMARY KEY Автоматическое увеличение на 1.	Код адреса, автовычисляемое поле
Фамилия	varchar(50)	NOT NULL	Фамилия работника
Имя	varchar(50)	NOT NULL	Имя работника
Отчество	varchar(50)	NULL	Отчество работника (при наличии)
id_должности_работника	INT	FOREIGN KEY, ON DELETE CASCADE NOT NULL	ВК, связь: Должность работника (id_должности_работника)
id_адреса	INT	FOREIGN KEY,	ВК, связь: Адрес (id_адреса)

		ON DELETE CASCADE NOT NULL	
id_паспорта	INT	FOREIGN KEY, ON DELETE CASCADE NOT NULL	ВК, связь: Паспорт (id_паспорта)
id_пользователя	INT	FOREIGN KEY, ON DELETE CASCADE NOT NULL	ВК, связь: Пользователь (id_пользователя)
Телефон	varchar(12)	NOT NULL Формат: 79991112233 Без других символов.	Рабочий номер телефона
Фото_работника	varchar (255)	NOT NULL	Ссылка на фотографию, размещённую на сервере
Эл_почта	varchar(50)	NOT NULL Формат: <u>название@доме</u> <u>н.домен</u>	Электронная почта

Описание таблицы 8:

Тип данных: Поле «id_работника» относится к типу данных – INT с атрибутом IDENTITY. Поля «id_должности_работника», «id_адреса», «id_паспорта», «id_пользователя» - int. Поля «Фамилия», «Имя», «Отчество», «Телефон», «Фото_работника», «Эл_почта» относятся к символьному типу, где varchar представляет строку переменной длины, а в скобках указывается длина хранимой строки.

Ограничения: Поле «id_работника» имеет ограничения PRIMARY KEY - первичный ключ, автоматическое увеличение на 1. Поля «id_должности_работника», «id_адреса», «id_паспорта», «id_пользователя» имеет ограничения FOREIGN KEY- внешний ключ, ON DELETE CASCADE- удаляет дочерние строки при удалении

родительского ключа. Поля «Фамилия», «Имя», «Телефон», «Фото_работника», «Эл_почта» должны быть обязательно заполненными (NOT NULL). Поле «Отчество» не обязательно.

Таблица 9 – Таблица «Работодатель»

Имя поля	Тип данных	Ограничения	Описание
id_работодателя	INT с атрибутом IDENTITY	PRIMARY KEY Автоматическое увеличение на 1.	Код адреса, автовычисляемое поле
Наименование_компании	varchar(50)	NOT NULL	Наименование компании
id_адреса	INT	FOREIGN KEY, ON DELETE CASCADE NOT NULL	ВК, связь: Адрес (id_адреса)
Телефон_компании	varchar(12)	NOT NULL Формат: 79991112233 Без других символов.	Рабочий номер телефона
Почта_компании	varchar(50)	NOT NULL Формат: <u>название@домени.домен</u>	Электронная почта
Информация_о_компании	varchar(50)	NOT NULL	Краткая информация о компании
Логотип_компании	varchar(255)	NOT NULL	Ссылка на логотип, размещённый на сервере
id_пользователя	INT	FOREIGN KEY, ON DELETE CASCADE NOT NULL	ВК, связь Пользователь (id_пользователя)

Счёт_компании	varchar(10)	NOT NULL Значение>0	Счёт компании
---------------	-------------	------------------------	---------------

Описание таблицы 9:

Тип данных: Поле «id_работодателя» относится к типу данных – INT с атрибутом IDENTITY. Поля «id_адреса», «id_пользователя» - int. Поля «Наименование_компании», «Телефон_компании», «Почта_компании», «Информация_о_компании», «Логотип_компании», «Счёт_компании» относятся к символьному типу, где varchar представляет строку переменной длины, а в скобках указывается длина хранимой строки.

Ограничения: Поле «id_работодателя» имеет ограничения PRIMARY KEY - первичный ключ, автоматическое увеличение на 1. Поля «id_адреса», «id_пользователя» имеет ограничения FOREIGN KEY- внешний ключ, ON DELETE CASCADE- удаляет дочерние строки при удалении родительского ключа. Поля «Наименование_компании», «Телефон_компании», «Почта_компании», «Информация_о_компании», «Логотип_компании», «Счёт_компании» должны быть обязательно заполненными (NOT NULL).

Таблица 10 – Таблица «Образование»

Имя поля	Тип данных	Ограничения	Описание
id_образования	INT с атрибутом IDENTITY	PRIMARY KEY Автоматическое увеличение на 1.	Код адреса, автовычисляемое поле
Уровень_образования	varchar(50)	NOT NULL	Уровень образования
Направление_образования	varchar(50)	NOT NULL	Направление образования

Описание таблицы 10:

Тип данных: Поле «id_образования» относится к типу данных – INT с атрибутом IDENTITY. Поля «Уровень_образования», «Направление_образования» относятся к символьному типу, где varchar представляет строку переменной длины, а в скобках указывается длина хранимой строки.

Ограничения: Поле «id_образования» имеет ограничения PRIMARY KEY - первичный ключ, автоматическое увеличение на 1. Поля «Уровень_образования», «Направление_образования» должны быть обязательно заполненными (NOT NULL).

Таблица 11 – Таблица «Вид деятельности»

Имя поля	Тип данных	Ограничения	Описание
id_вида_деятельности	INT с атрибутом IDENTITY	PRIMARY KEY Автоматическое увеличение на 1.	Код адреса, автовычисляемое поле
Наименование_вида_деятельности	varchar(50)	NOT NULL	Наименование вида деятельности

Описание таблицы 11:

Тип данных: Поле «id_вида_деятельности» относится к типу данных – INT с атрибутом IDENTITY. Поле «Наименование_вида_деятельности» относится к символьному типу, где varchar представляет строку переменной длины, а в скобках указывается длина хранимой строки.

Ограничения: Поле «id_вида_деятельности» имеет ограничения PRIMARY KEY - первичный ключ, автоматическое увеличение на 1. Поле «Наименование_вида_деятельности» должны быть обязательно заполненными (NOT NULL).

Таблица 12 – Таблица «Вакансия»

Имя поля	Тип данных	Ограничения	Описание
id_вакансия	INT с атрибутом IDENTITY	PRIMARY KEY Автоматическое увеличение на 1.	Код адреса, автовычисляемое поле
Наименование	varchar(50)	NOT NULL	Наименование вакансии
id_вида_деятельности	INT	FOREIGN KEY, ON DELETE CASCADE NOT NULL	ВК, связь: Вид деятельности (id_вида деятельности)

Уровень 3/П	varchar(10)	NOT NULL Значение>0	Уровень заработной платы
id_образования	INT	FOREIGN KEY, ON DELETE CASCADE NOT NULL	ВК, связь: Образование (id_образования)
id_работодателя	INT	FOREIGN KEY, ON DELETE CASCADE NOT NULL	ВК, связь: Работодатель (id_работодатель)

Описание таблицы 12:

Тип данных: Поле «id_вакансия» относится к типу данных – INT с атрибутом IDENTITY. Поля «id_вида деятельности», «id_образования», «id_работодателя» - int. Поля «Наименование», «Уровень 3/П» относятся к символьному типу, где varchar представляет строку переменной длины, а в скобках указывается длина хранимой строки.

Ограничения: Поле «id_вакансия» имеет ограничения PRIMARY KEY - первичный ключ, автоматическое увеличение на 1. Поля «id_вида деятельности», «id_образования», «id_работодателя» имеет ограничения FOREIGN KEY- внешний ключ, ON DELETE CASCADE- удаляет дочерние строки при удалении родительского ключа. Поля «Наименование», «Уровень 3/П» должны быть обязательно заполненными (NOT NULL).

Таблица 13 – Таблица «Соискатель»

Имя поля	Тип данных	Ограничения	Описание
id_соискателя	INT с атрибутом IDENTITY	PRIMARY KEY Автоматическое увеличение на 1.	Код адреса, автовычисляемое поле
Фамилия	varchar(50)	NOT NULL	Фамилия соискателя
Имя	varchar(50)	NOT NULL	Имя соискателя
Отчество	varchar(50)	NULL	Отчество соискателя (при наличии)

Телефон	varchar(12)	NOT NULL Формат: 79991112233 Без других символов.	Рабочий номер телефона
Почта	varchar(50)	NOT NULL Формат: <u>название@доме</u> <u>н.домен</u>	Электронная почта
Информация о себе	varchar(50)	NOT NULL	Краткая информация о себе
Фото	varchar(255)	NOT NULL	Ссылка на логотип, размещённый на сервере
id_вида деятельности	INT	FOREIGN KEY, ON DELETE CASCADE NOT NULL	ВК, связь: Вид деятельности (id_вида деятельности)
Желаемая з/п	varchar(10)	NOT NULL Значение>0	Желаемый уровень дохода
id_образования	INT	FOREIGN KEY, ON DELETE CASCADE NOT NULL	ВК, связь: Образование (id_образования)
id_пользователя	INT	FOREIGN KEY, ON DELETE CASCADE NOT NULL	ВК, связь Пользователь (id_пользователя)

Описание таблицы 13:

Тип данных: Поле «id_соискателя» относится к типу данных – INT с атрибутом IDENTITY. Поля «id_вида деятельности», «id_образования», «id_пользователя» - int. Поля «Фамилия», «Имя», «Отчество», «Телефон», «Почта». «Информация о себе», «Фото», «Желаемая з/п» относятся к символьному типу, где varchar представляет строку переменной длины, а в скобках указывается длина хранимой строки.

Ограничения: Поле «id_соискателя» имеет ограничения PRIMARY KEY - первичный ключ, автоматическое увеличение на 1. Поля id_вида деятельности», «id_образования», «id_пользователя» имеет ограничения FOREIGN KEY- внешний ключ, ON DELETE CASCADE - удаляет дочерние строки при удалении родительского ключа. Поля «Фамилия», «Имя», «Телефон», «Почта». «Информация о себе», «Фото», «Желаемый уровень дохода» должны быть обязательно заполненными (NOT NULL). Поле «Отчество» не обязательно.

Таблица 14 – Таблица «Сделка»

Имя поля	Тип данных	Ограничения	Описание
id_сделки	INT с атрибутом IDENTITY	PRIMARY KEY Автоматическое увеличение на 1.	Код адреса, автовычисляемое поле
id_работника	INT	FOREIGN KEY, ON DELETE CASCADE NOT NULL	ВК, связь: Работник (id_работника)
id_работодателя	INT	FOREIGN KEY, ON DELETE CASCADE NOT NULL	ВК, связь Работодатель (id_работодателя)
id_соискателя	INT	FOREIGN KEY, ON DELETE CASCADE NOT NULL	ВК, связь Соискатель (id_соискателя)
Дата сделки	Date	NOT NULL Формат: YYYY-MM-DD	Дата совершенной сделки
Стоимость сделки	varchar(10)	NOT NULL Значение>0	Стоимость совершенной сделки
id_вакансии	INT	FOREIGN KEY,	ВК, связь Вакансия (id_вакансии)

		ON DELETE CASCADE NOT NULL	
--	--	----------------------------------	--

Описание таблицы 14:

Тип данных: Поле «id_сделки» относится к типу данных – INT с атрибутом IDENTITY. Поля «id_работника», «id_работодателя», «id_соискателя», «id_вакансии» - int. Поле «Дата сделки» - date. Поле «Стоимость сделки» относится к символьному типу, где varchar представляет строку переменной длины, а в скобках указывается длина хранимой строки.

Ограничения: Поле «id_сделки» имеет ограничения PRIMARY KEY - первичный ключ, автоматическое увеличение на 1. Поля «id_работника», «id_работодателя», «id_соискателя», «id_вакансии» имеет ограничения FOREIGN KEY- внешний ключ, ON DELETE CASCADE - удаляет дочерние строки при удалении родительского ключа. Поля «Дата сделки», «Стоимость сделки» должны быть обязательно заполненными (NOT NULL).

2.3.4. Роли и права доступа к данным

В данной базе данных имеется 4 роли – Администратор, Работник бюро, Соискатель и Работодатель. Подлинность проверки SQL Server используется как вход на сервер под своим именем входа и паролем.

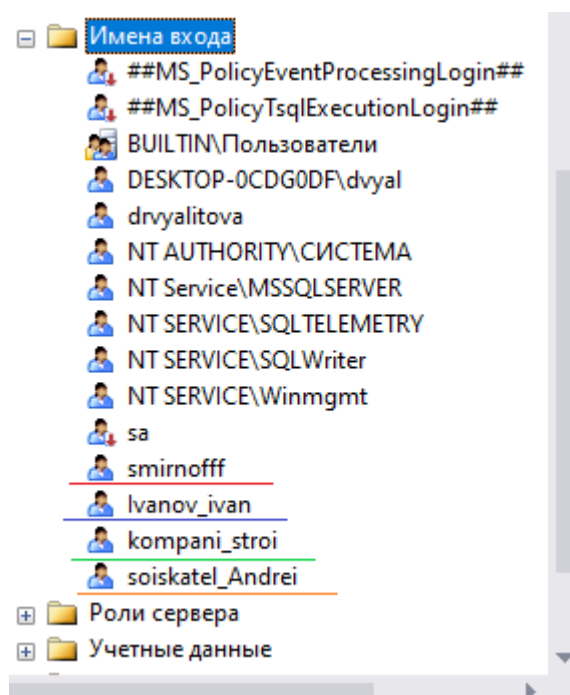


Рисунок 14 – Имена входов

На рисунке 14 красной линией показан «Администратор», синей – «Работник бюро», зеленой – «Работодатель», оранжевой – «Соискатель». Во вкладке пользователи аналогичные имена.

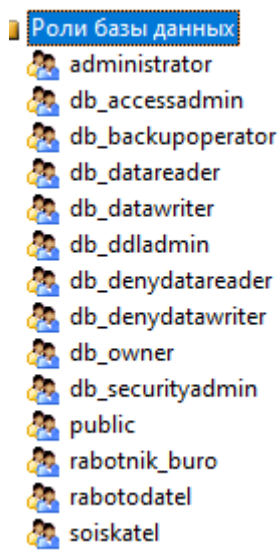


Рисунок 15 – Роли базы данных

2.4. Проектирование интерфейса клиентской части

От «Администратора»:

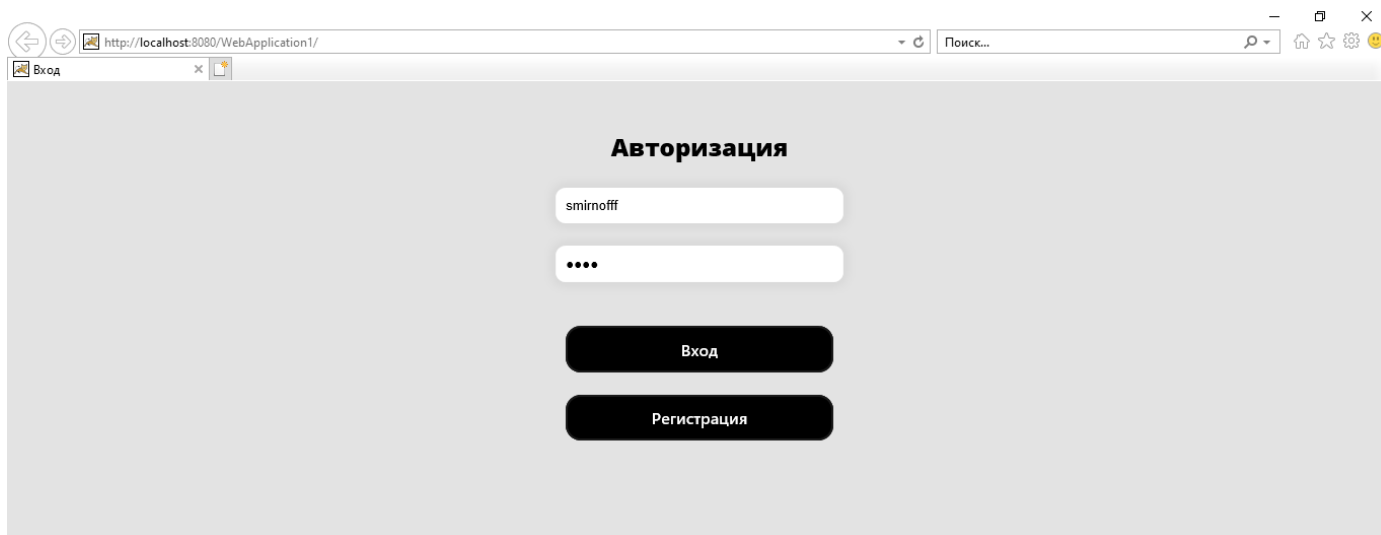


Рисунок 16 – Авторизация от администратора

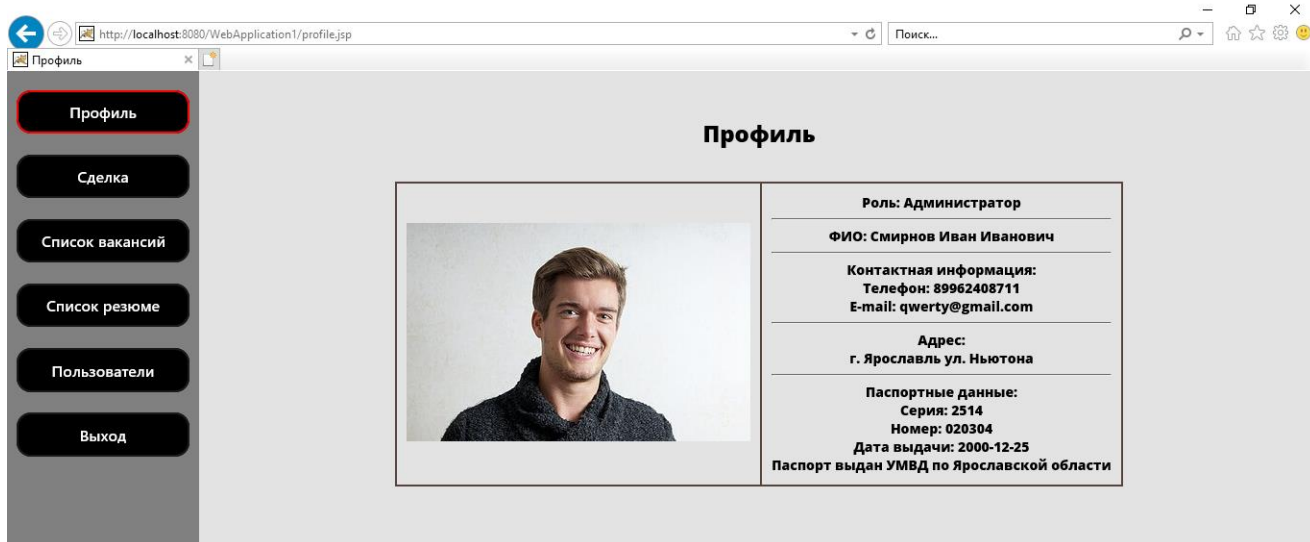


Рисунок 17 – Профиль администратора

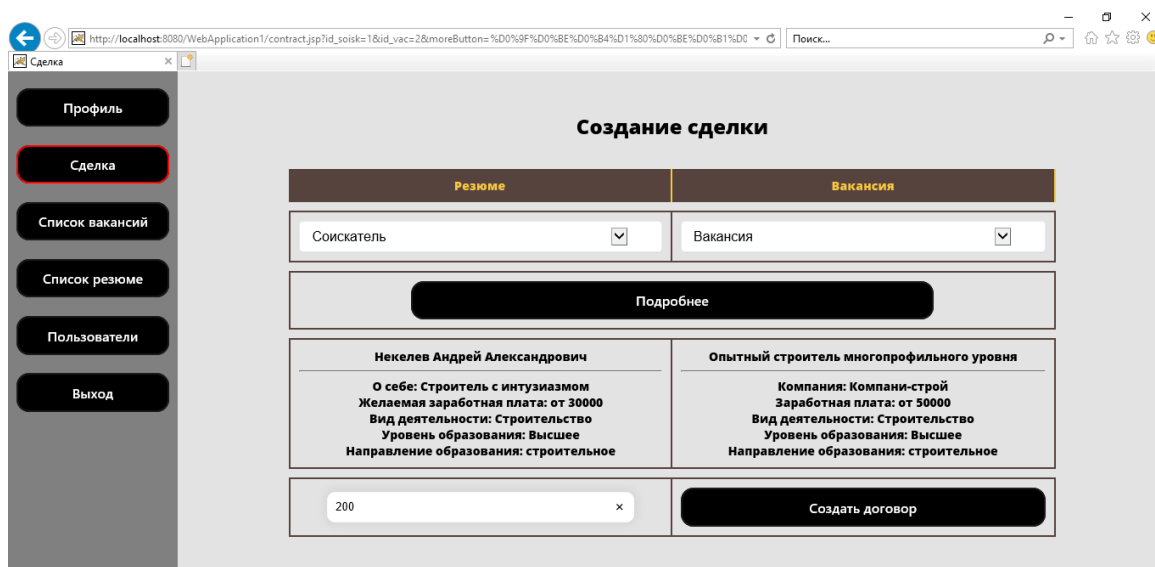


Рисунок 18 – Создание сделки

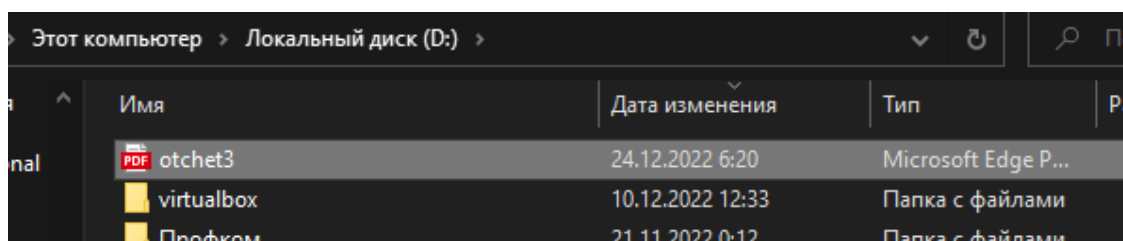


Рисунок 19 – Созданный договор

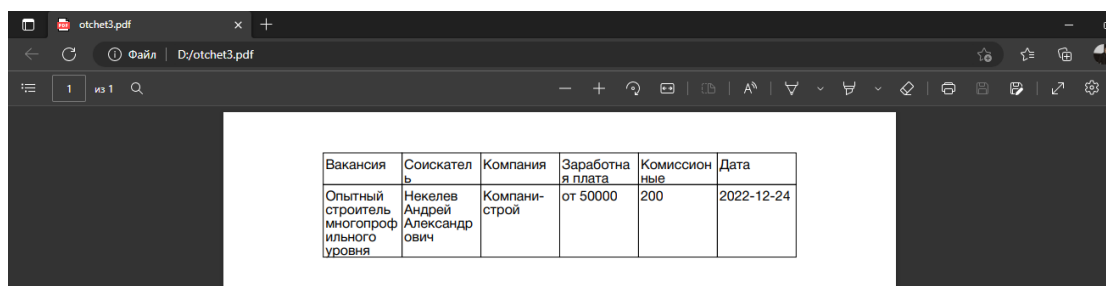


Рисунок 20 – Содержание договора

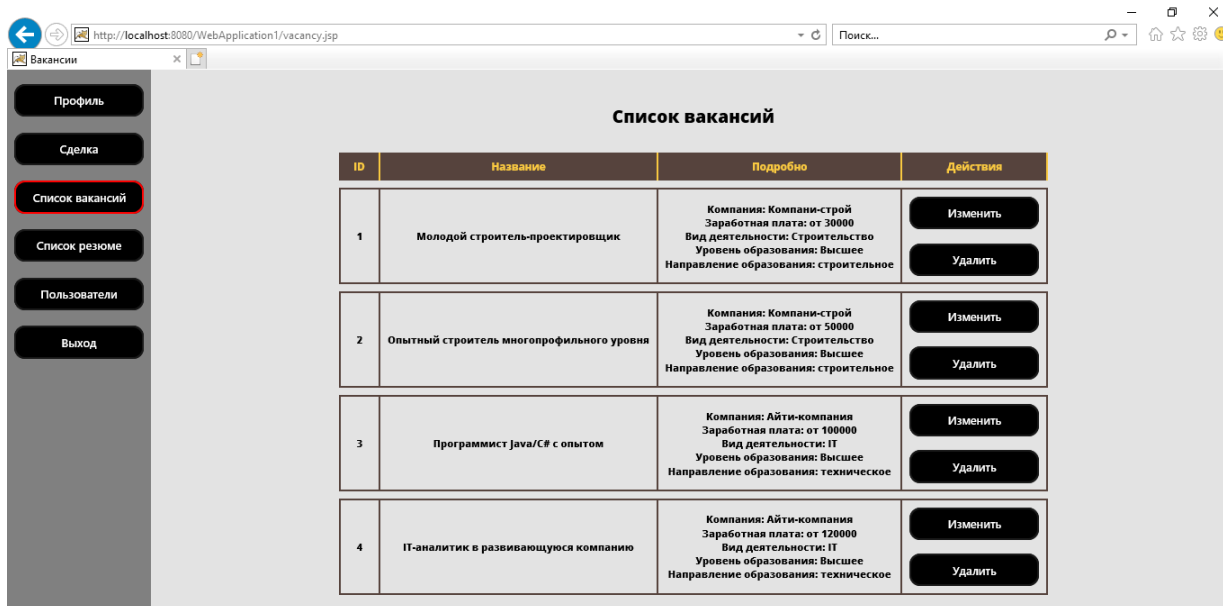


Рисунок 21 – Список вакансий

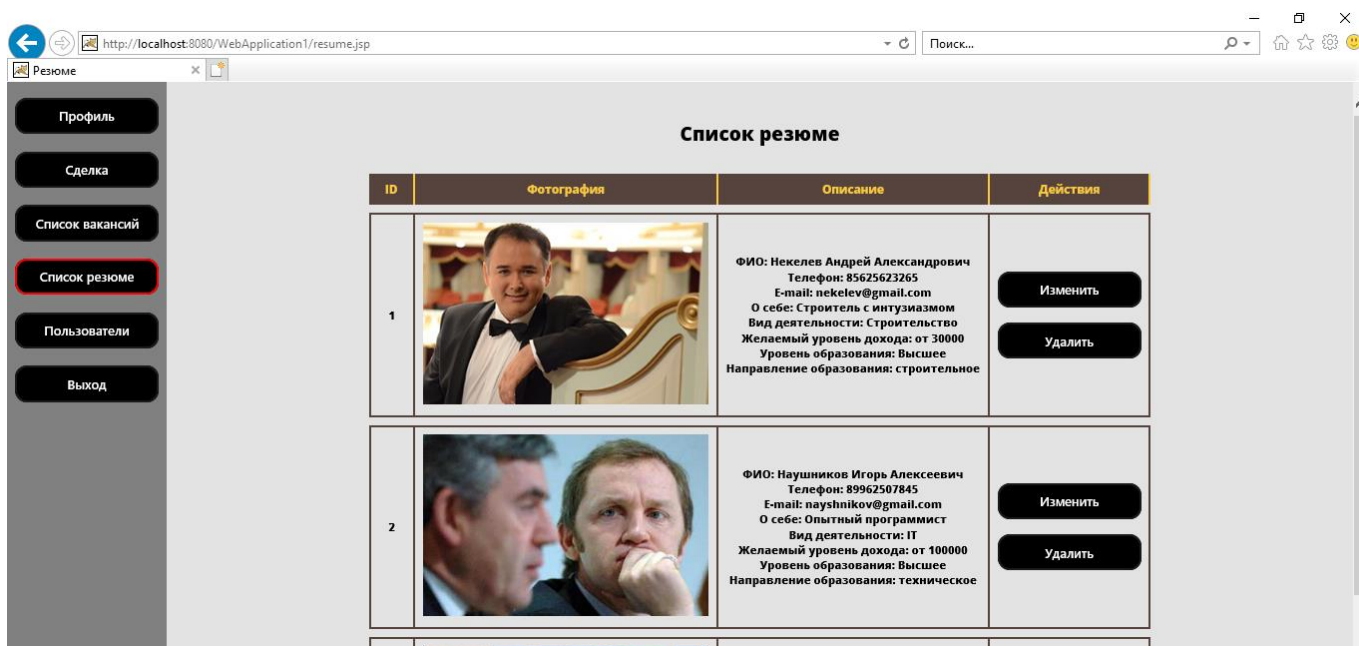


Рисунок 22 – Список резюме

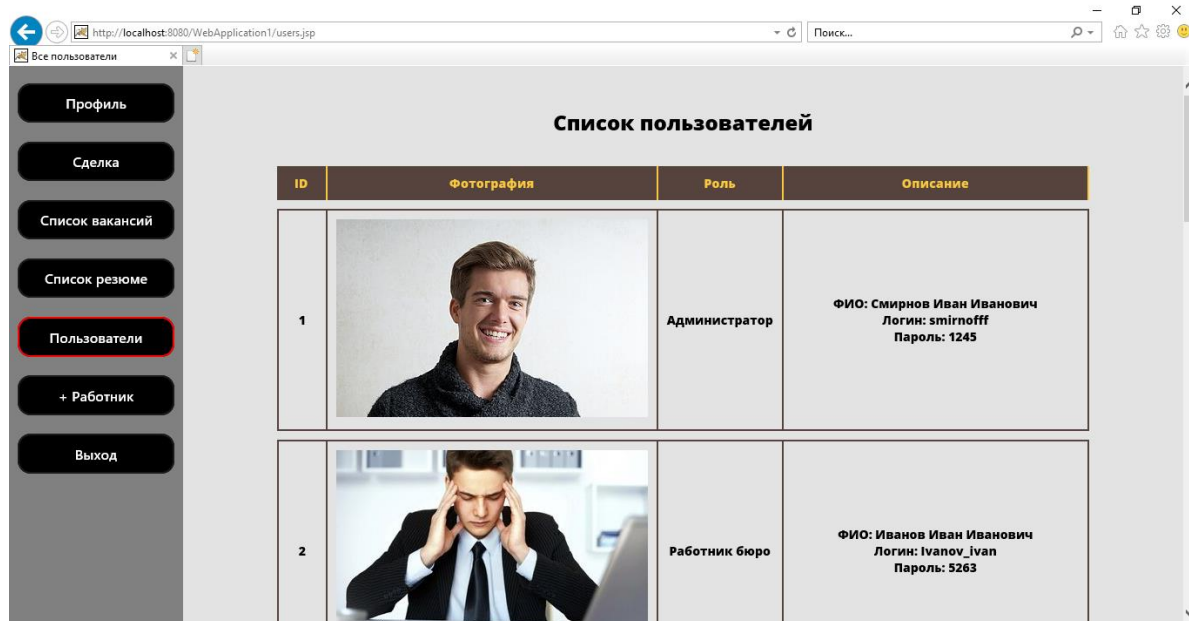


Рисунок 23 – Список пользователей

Данные работника бюро

Работников

Работник

Работникович

Агент бюро

1245

206356

20.12.2022

УМВД по Ярославской области

89965208965

robotnik@gmail.ru

Рисунок 24 – Создание работника

11		Работник бюро	ФИО: Работников Работник Работникович Логин: rab123 Пароль: 123
----	---	---------------	---

Рисунок 25 – Созданный работник

От «Работника бюро»:

The screenshot shows a web page titled "Авторизация" (Authorization). It features two input fields: the first contains the text "работник1", and the second contains four asterisks "****". Below these fields are two buttons: "Вход" (Login) and "Регистрация" (Registration).

Рисунок 25 – Авторизация от работника бюро

The screenshot shows a web browser window with the URL "http://localhost:8080/WebApplication1/profile.jsp". The page is titled "Профиль" (Profile). On the left is a sidebar with buttons: "Профиль", "Сделка", "Список вакансий", "Список резюме", and "Выход". The main content area displays a profile for a bureau employee. It includes a photo of a man in a suit and a table with the following information:

Роль: Работник бюро
ФИО: Мандаринов Михаил Александрович
Контактная информация: Телефон: 8996896552 E-mail: mandarinov@gmail.com
Адрес: г. Ярославль ул. Московская
Паспортные данные: Серия: 7856 Номер: 859652 Дата выдачи: 2007-08-26 Паспорт выдан УМВД по Ярославской области

Рисунок 26 – Профиль работника бюро

The screenshot shows a web browser window with the URL "http://localhost:8080/WebApplication1/contract.jsp?id_soisk=2&id_vac=1&moreButtons=%D0%9F%D0%BE%D0%B4%D1%80%D0%BE%D0%B1%DC". The page is titled "Создание сделки" (Deal Creation). On the left is a sidebar with buttons: "Профиль", "Сделка", "Список вакансий", "Список резюме", and "Выход". The main content area has a header with "Резюме" and "Вакансия" tabs. Below this are two dropdown menus: "Соискатель" (selected) and "Программист Java/C# с опытом" (selected). A "Подробнее" button is below these. The main area is divided into two columns. The left column contains information about a candidate: "Наушников Игорь Алексеевич", "О себе: Опытный программист", "Желаемая заработная плата: от 100000", "Вид деятельности: IT", "Уровень образования: Высшее", and "Направление образования: техническое". The right column contains information about a vacancy: "Молодой строитель-проектировщик", "Компания: Компани-строй", "Заработная плата: от 30000", "Вид деятельности: Строительство", "Уровень образования: Высшее", and "Направление образования: строительное". At the bottom, there is a text input field with "100" and a "Создать договор" button.

Рисунок 27 – Создание сделки

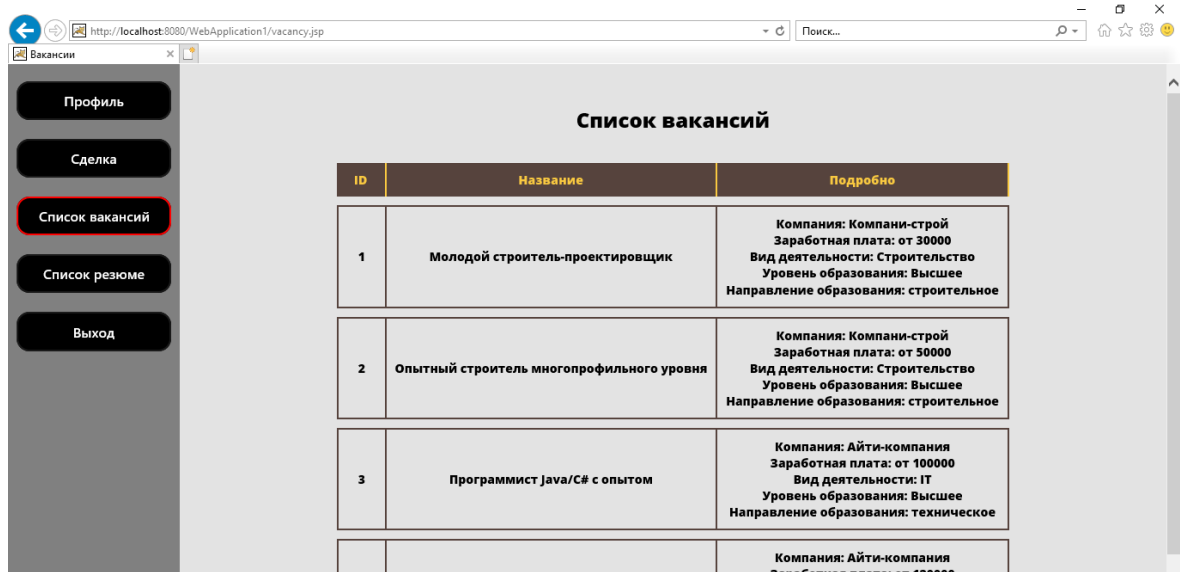


Рисунок 28 – Список вакансий

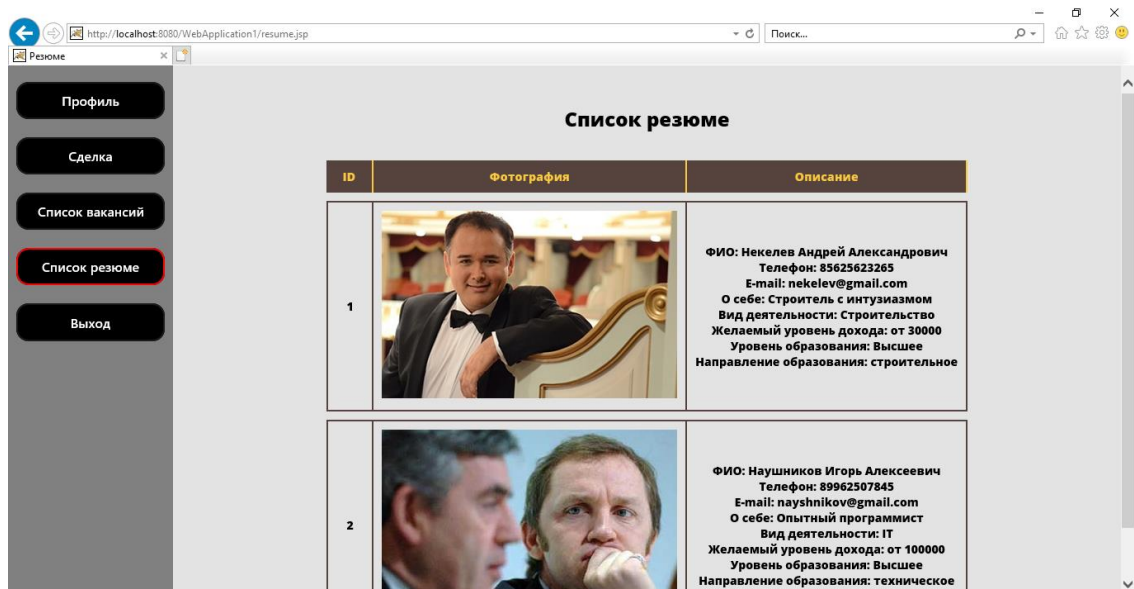


Рисунок 29 – Список резюме

От «Работодателя»:

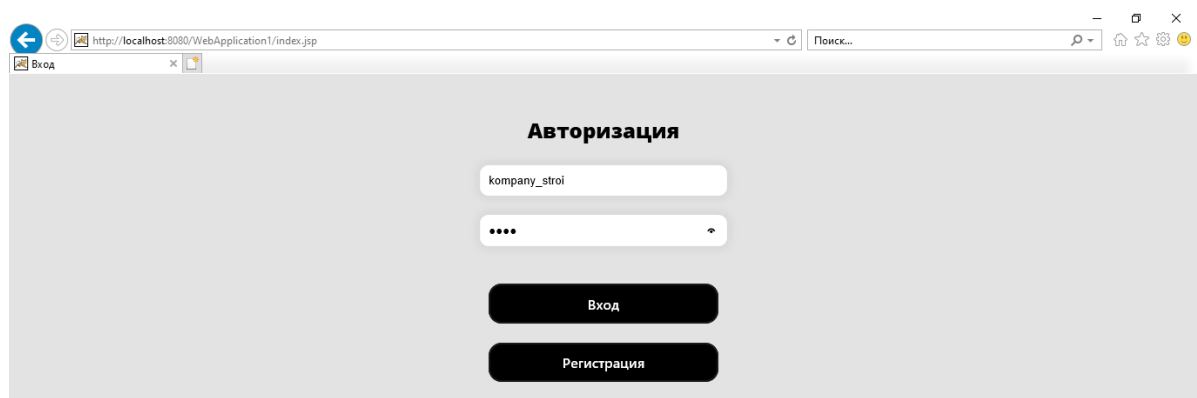


Рисунок 30 – Авторизация от работодателя

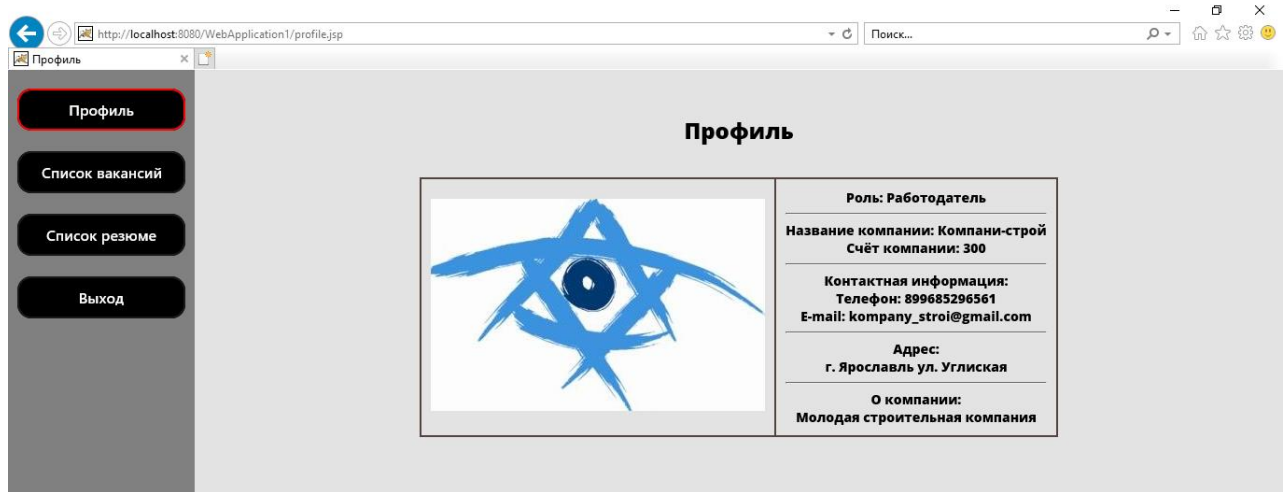


Рисунок 31 – Профиль работодателя

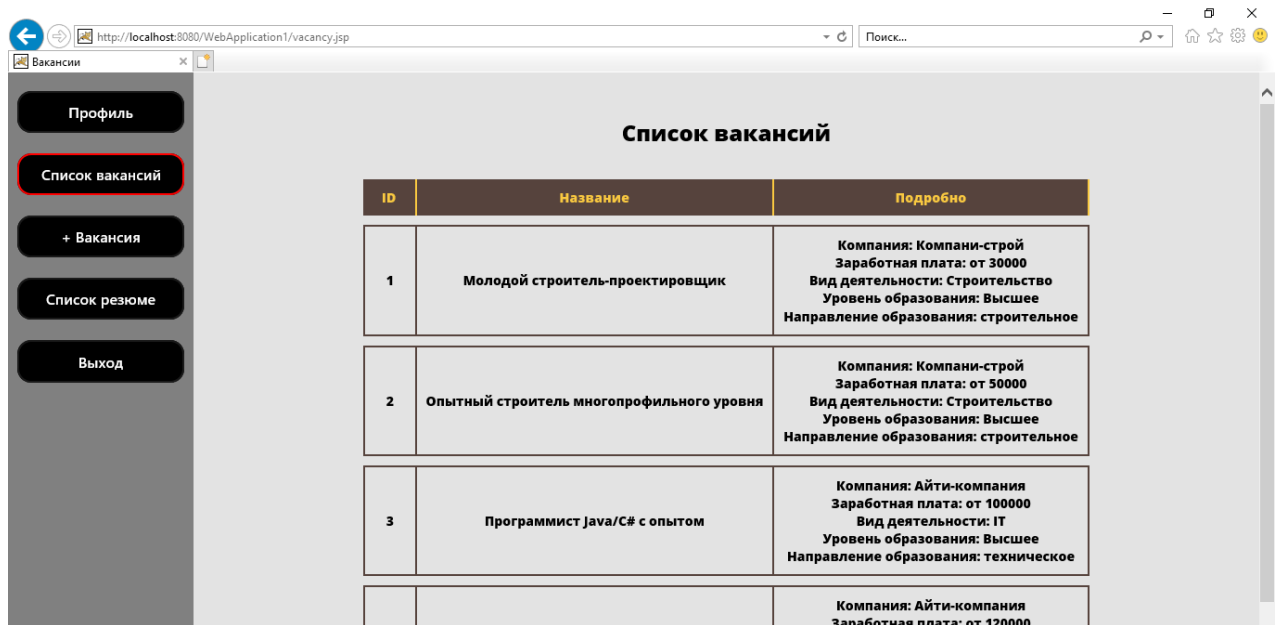


Рисунок 32 – Список вакансий

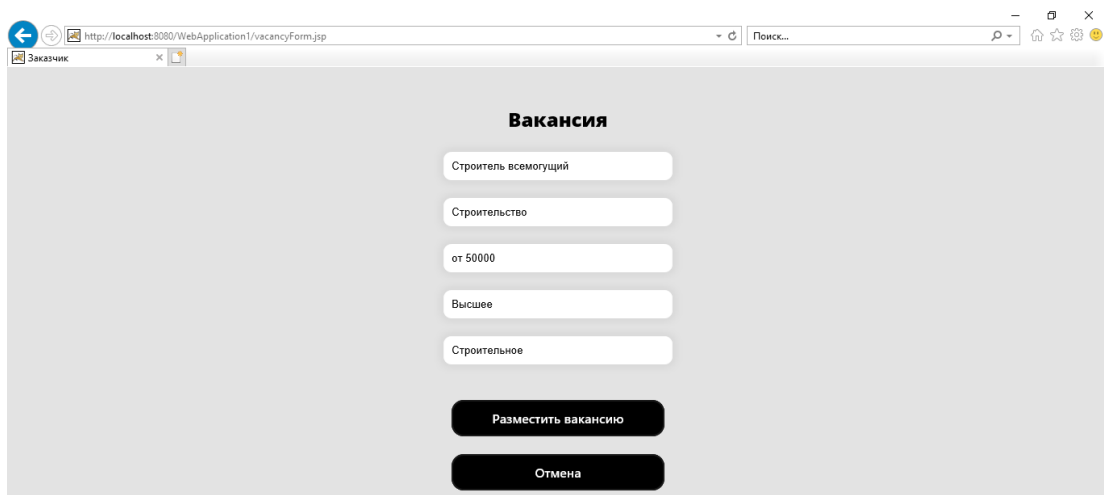


Рисунок 33 – Размещение вакансии

6	Строитель всемогущий	Компания: Компани-строй Заработная плата: от 50000 Вид деятельности: Строительство Уровень образования: Высшее Направление образования: Строительное
---	----------------------	---

Рисунок 34 – Вакансия размещена

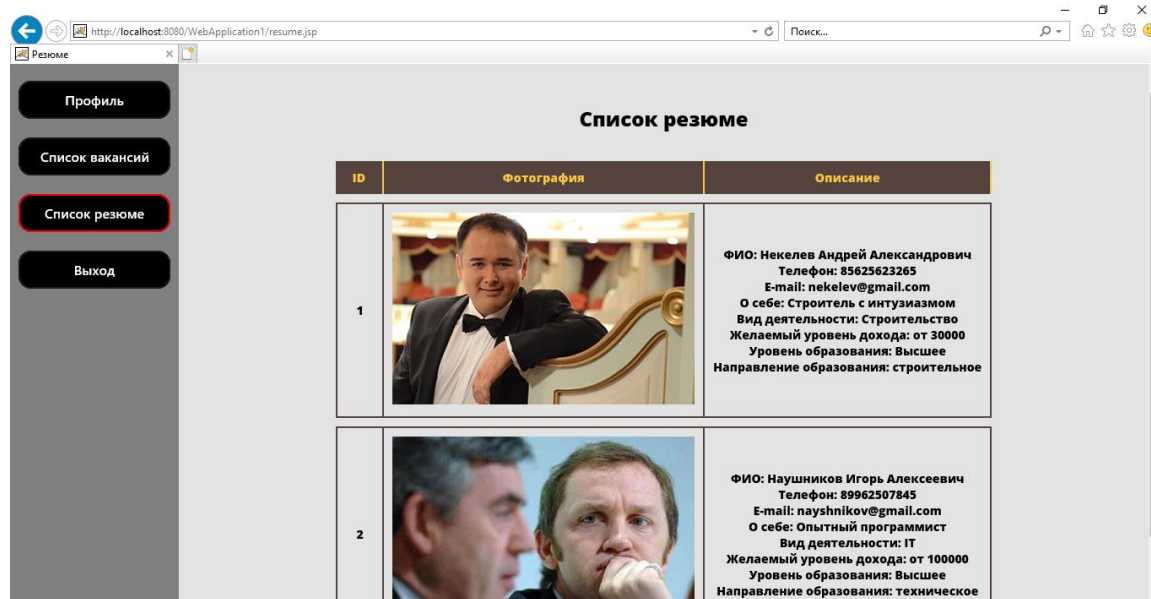


Рисунок 35 – Список резюме

От «Соискателя»:

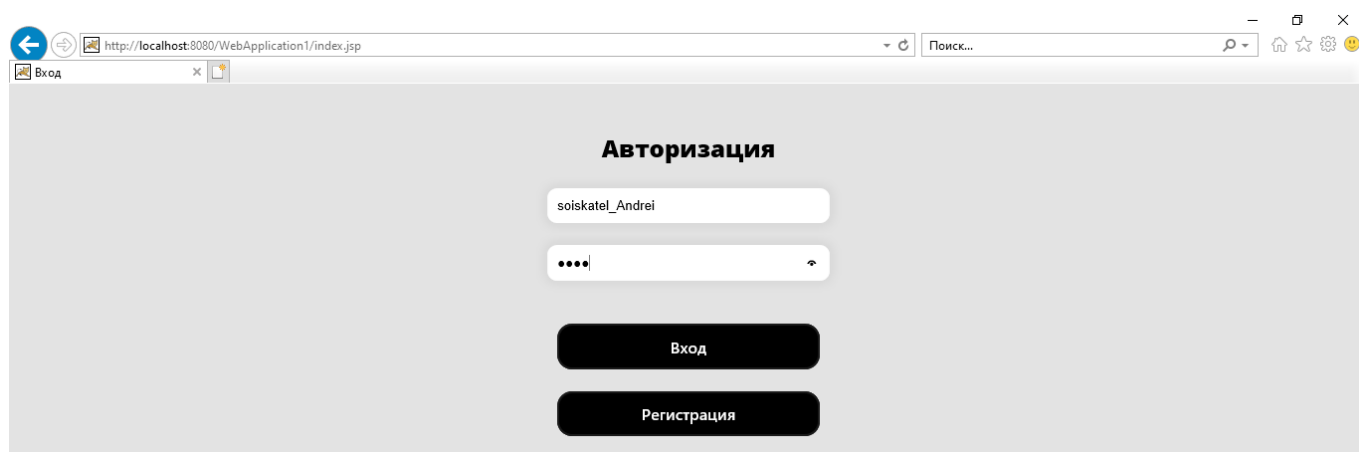


Рисунок 36 – Авторизация соискателя

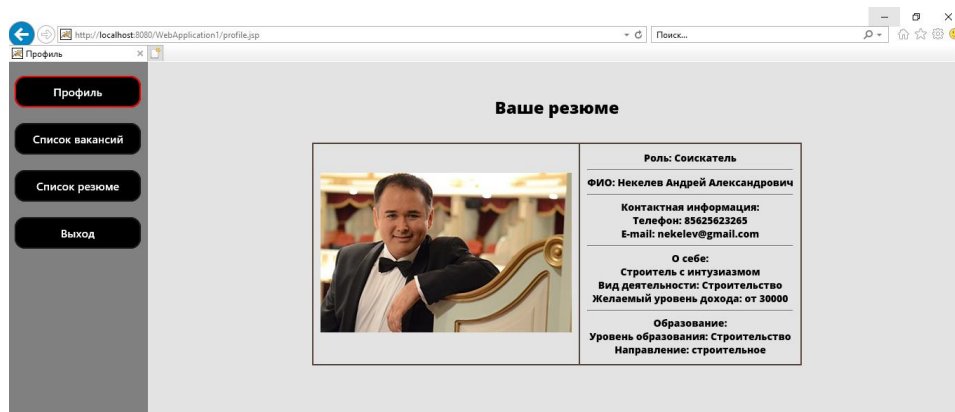


Рисунок 37 – Резюме соискателя

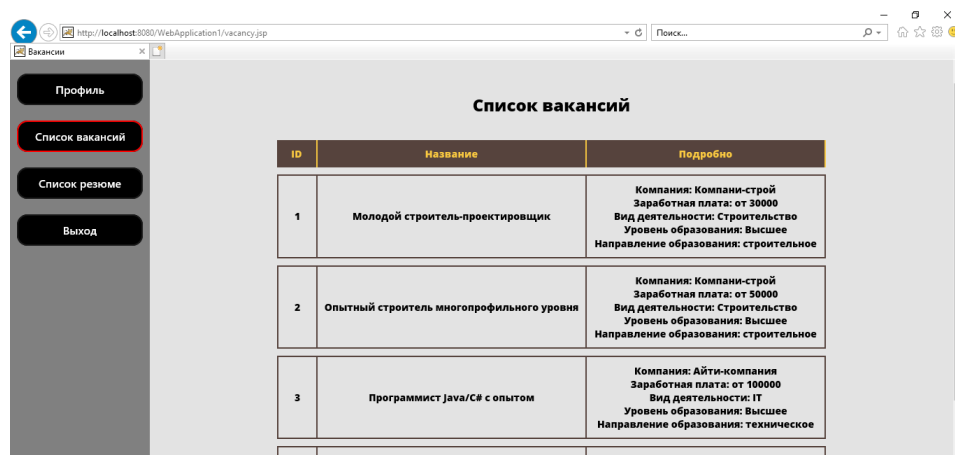


Рисунок 38 – Список вакансий

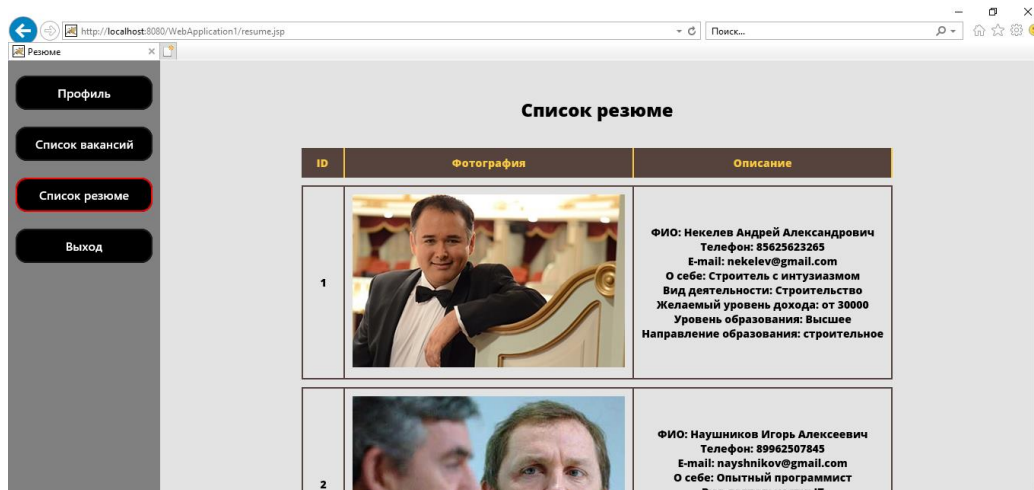


Рисунок 39 – Список резюме

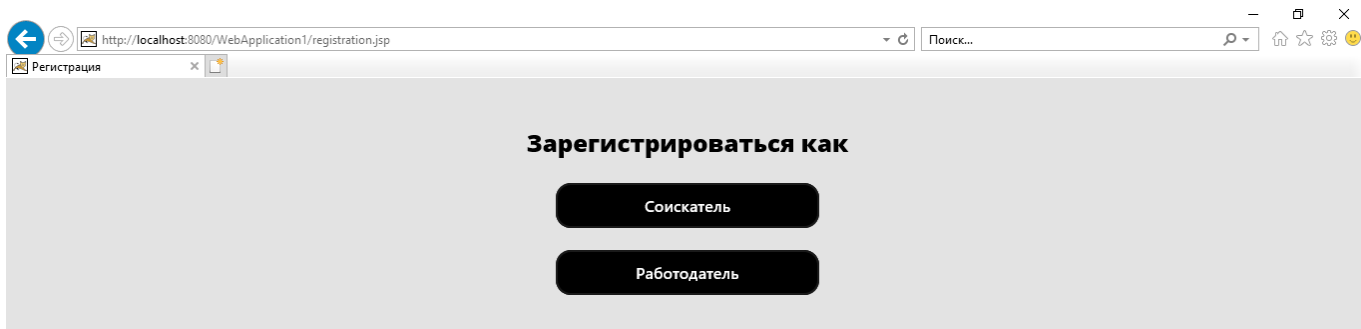


Рисунок 40 – Регистрация в приложении

Заполните резюме

Одеялов

Одеял

Одеялович

89965203265

odeyalo@gmail.ru

Добрый, хороший

rabotnik6.jpg

Ничего

от 50000

Среднее

Рисунок 41 – Регистрация соискателя (она же – заполнение резюме)

Ваше резюме

	Роль: Соискатель
	ФИО: Одеялов Одеял Одеялович
	Контактная информация: Телефон: 89965203265 E-mail: odeyalo@gmail.ru
	О себе: Добрый, хороший Вид деятельности: Ничего Желаемый уровень дохода: от 50000
	Образование: Уровень образования: Ничего Направление: -

Рисунок 41 – Зарегистрированный пользователь

Рисунок 42 – Регистрация работодателя

Роль: Работодатель
Название компании: Олимп
Счёт компании: 1000
Контактная информация:
Телефон: 86523259652
E-mail: olimp@gmail.ru
Адрес:
г. Ярославль ул. Кривова д.2
О компании:
Зарабатываем реальные деньги

Рисунок 43 – Зарегистрированный пользователь

3. Технологическая часть

3.1. Выбор программных средств

3.1.1. Выбор СУБД

Microsoft SQL Server — система управления реляционными базами данных (РСУБД), разработанная корпорацией Microsoft. Основным используемый язык запросов — Transact-SQL. Используется для работы с базами данных размером от персональных до крупных баз данных масштаба предприятия. SQL Server Management Studio (SSMS) — утилита из Microsoft SQL Server 2005 и более поздних версий для конфигурирования, управления и администрирования всех компонентов Microsoft SQL Server. Утилита включает скриптовый редактор и графическую программу, которая работает с объектами и настройками сервера

3.1.2. Выбор средств разработки

NetBeans – мощная бесплатная интегрированная среда разработки приложений (IDE). Поддерживает языки программирования Java, PHP, JavaScript, Ajax, Ruby, Ruby on Rails, C/C++ и другие. Изначально NetBeans позиционировался, как стандартный IDE для Java, но постепенно начал поддерживать все больше языков программирования. Функциональность программы может быть расширена с помощью массы доступных плагинов. Netbeans поддерживает работу с системами контроля версий. Позволяет просматривать изменения между локальной копией и копией файла в хранилище. Подсвечивает новые и измененные файлы. Позволяет фиксировать (коммитить) изменения, добавлять новые файлы и так далее. Обладает достаточным набором средств для работы с системами контроля версий, что позволяет не использовать для этого командную строку или сторонние приложения.

Возможности NetBeans:

- а) Поддержка проектов;
- б) Автодополнение кода;
- в) Продвинутая навигация по коду;
- г) Индикация ошибок уже на этапе редактирования кода («на лету»);
- д) Встроенная отладка;
- е) Поддержка систем контроля версий: GIT, CVS, SVN, MERCURIAL, FTP, SFTP;
- ж) Поддержка плагинов;
- з) Автоматическая установка плагинов;
- и) и многое другое.

3.2. Описание работы с программой

Пользователь с ролью «Администратор»:

- 1) Авторизация по логину и паролю для входа в систему.
- 2) Администратор может выполнять все функции, поэтому ему доступны все действия. Он выбирает пункт меню, заходит на определенную им страницу. Все страницы работают аналогично друг другу.
- 3) Для просмотра данных администратор выбирает нужную ему страницу, где находится таблица со всеми данными.
- 4) Для добавления данных ему надо нажать на новую появившуюся кнопку меню черного цвета. На каждой странице кнопка аналогична. Открывается страница с формой для заполнения данных. После заполнения надо нажать кнопку «Добавить», и данные сохранятся в базе данных. Если надо выйти, нажимаем кнопку «Отмена». После добавления данных происходит переход на предыдущую страницу.
- 5) Для редактирования данных, которые необходимо изменить, нужно нажать кнопку «Изменить». Открывается страница с уже заполненной формой. Администратор выбирает нужное ему поле и изменяет его. Для сохранения изменений необходимо нажать на кнопку «Сохранить», и данные изменятся в базе

данных. Если надо выйти, нажимаем кнопку «Отмена». После сохранения данных происходит переход на предыдущую страницу.

6) Для удаления данных, которые необходимо удалить, нужно нажать кнопку «Удалить». Происходит удаление данных из базы данных.

7) Для выхода из приложения надо выбрать пункт меню «Выход».

Пользователь с ролью «Работник бюро»:

- 1) Авторизация по логину и паролю для входа в систему.
- 2) Работник бюро может выполнять не все функции, поэтому ему некоторые действия не доступны. Он выбирает пункт меню, заходит на определенную им страницу. Все страницы работают аналогично друг другу. Пункта «Пользователи», «Добавить работника» у него нет.
- 3) Для просмотра данных работник бюро выбирает нужную ему страницу, где находится таблица со всеми данными.
- 4) Работник бюро может просматривать резюме и вакансии.
- 5) Работник бюро может заключать сделки между соискателем и работодателем во вкладке сделка по форме.
- 6) Для выхода из приложения надо выбрать пункт меню «Выход».

Пользователь с ролью «Соискатель»:

- 1) Авторизация по логину и паролю для входа в систему.
- 2) При регистрации автоматически создается резюме;
- 3) Пользователь может только просматривать определенные данные, поэтому ему некоторые действия не доступны. Он выбирает пункт меню, заходит на определенную им страницу. Все страницы работают аналогично друг другу. Пункта «Добавить работника», «Добавить вакансию», «Пользователи», «Сделка» у него нет.
- 4) Для просмотра данных пользователь выбирает нужную ему страницу, где находится таблица со всеми данными.
- 5) Для выхода из приложения надо выбрать пункт меню «Выход».

Пользователь с ролью «Работодатель»:

- 1) Авторизация по логину и паролю для входа в систему.
- 2) Пользователь может разместить несколько вакансий;
- 3) Пользователь может только просматривать определенные данные, поэтому ему некоторые действия не доступны. Он выбирает пункт меню, заходит на определенную им страницу. Все страницы работают аналогично друг другу. Пункта «Добавить работника», «Пользователи», «Сделка» у него нет.
- 4) Для просмотра данных пользователь выбирает нужную ему страницу, где находится таблица со всеми данными.
- 5) Для выхода из приложения надо выбрать пункт меню «Выход».

Заключение

При разработке проекта учитывались все особенности информационной системы.

В результате курсового проекта были решены следующие задачи:

- 1) Проанализирована предметная область;
- 2) Спроектирована схема базы данных;
- 3) Создана и заполнена БД;
- 4) Осуществлен удобный доступ к данным базы данных;
- 5) Разработано web-приложение для работы пользователей;
- 6) Создана возможность создания отчета, в котором будут храниться данные вакансии, соискателя, компании, з/п, комиссионные и дата.

Итогом курсовой работы стала система, позволяющая автоматизировать часть процессов работы бюро по трудоустройству.

Список использованных источников и литературы

- 1) Ткачев О.А. Создание и манипулирование базами данных средствами СУБД Microsoft SQL Server 2008 [Электронный ресурс]: Учебное пособие/ Ткачев О.А. – Электрон. текстовые данные. – М.: Московский городской педагогический университет, 2013. – 152 с. – Режим доступа: <http://www.bibliocomlectator.ru/book/?id=26613>
- 2) Чубукова И.А. Data Mining [Электронный ресурс]/ Чубукова И.А. – Электрон. текстовые данные. – М.: Интернет-Университет Информационных Технологий (ИНТУИТ), 2016. – 470 с. – Режим доступа: <http://www.bib1iocomp1ectator.ru/book/?id=56315>
- 3) [Электронный ресурс] – Режим доступа. - <https://metanit.com/java/> (дата посещения 09.11.2022)
- 4) [Электронный ресурс] – Режим доступа. - <https://habr.com/ru/post/326614/> (дата посещения 12.12.2022)
- 5) [Электронный ресурс] – Режим доступа. - <https://tproger.ru/articles/pishem-java-veb-prilozhenie-na-sovremennom-steke-s-nulja-do-mikroservisnoj-arhitektury-chast-1/> (дата посещения 20.12.2022)

Приложение А

Техническое задание

УТВЕРЖДАЮ

Преподаватель

_____ Шулева Ю.Н.

“ _____ ” _____ 20__

Программа для бюро по трудоустройству

Техническое задание

Лист утверждения

А.В.00001-01 ТЗ 01-ЛУ

дат а По дп ис	
ду бл № Ин в	
№ ин в Вз ам	
дат а По дп ис	
дл по № Ин в	

Руководитель разработки

_____ Шулева Ю.Н.

“ _____ ” _____ 20__

Исполнитель

_____ Вялитова Д.Р.

“ _____ ” _____ 20__

УТВЕРЖДЕНО

А.В.00001-01 ТЗ 01-ЛУ

**Программа для бюро по трудоустройству
Техническое задание**

А.В.00001-01 ТЗ 01-ЛУ

та да По дп ис	
д ду № И в	
№ ин в Вз а	
та да По дп ис	
дл по № И в	

АННОТАЦИЯ

В данном программном документе приведено техническое задание на разработку информационной подсистемы для бюро по трудоустройству.

В данном программном документе, в разделе «Введение» указано наименование, краткая характеристика области применения программы.

В разделе «Основания для разработки» указаны документы, на основании которых ведется разработка, наименование и условное обозначение темы разработки.

В данном программном документе, в разделе «Назначение разработки» указано функциональное и эксплуатационное назначение программы (программного изделия).

Раздел «Требования к программе» содержит следующие подразделы:

- а) требования к функциональным характеристикам;
- б) требования к надежности;
- в) условия эксплуатации;
- г) требования к составу и параметрам технических средств;
- д) требования к информационной и программной совместимости;
- е) специальные требования.

В данном программном документе, в разделе «Требования к программной документации» указаны предварительный состав программной документации и специальные требования к ней.

В разделе «Технико-экономические показатели» указаны: ориентировочная экономическая эффективность, экономические преимущества разработки.

В данном программном документе, в разделе «Стадии и этапы разработки» установлены необходимые стадии разработки, этапы и содержание работ.

В разделе «Порядок контроля и приемки» должны быть указаны виды испытаний и общие требования к приемке работы.

Оформление программного документа «Техническое задание» произведено по требованиям ЕСПД (ГОСТ 19.101-77 ¹⁾, ГОСТ 19.103-77 ²⁾, ГОСТ 19.104-78* ³⁾, ГОСТ 19.105-78* ⁴⁾, ГОСТ 19.106-78* ⁵⁾, ГОСТ 19.201-78 ⁶⁾, ГОСТ 19.604-78* ⁷⁾).

¹⁾ ГОСТ 19.101-77 ЕСПД. Виды программ и программных документов

²⁾ ГОСТ 19.103-77 ЕСПД. Обозначение программ и программных документов

³⁾ ГОСТ 19.104-78* ЕСПД. Основные надписи

⁴⁾ ГОСТ 19.105-78* ЕСПД. Общие требования к программным документам

⁵⁾ ГОСТ 19.106-78* ЕСПД. Общие требования к программным документам, выполненным печатным способом

⁶⁾ ГОСТ 19.201-78 ЕСПД. Техническое задание. Требования к содержанию и

оформлению ⁷⁾ ГОСТ 19.604-78* ЕСПД. Правила внесения изменений в программные документы, выполненные печатным способом

СОДЕРЖАНИЕ

АННОТАЦИЯ.....	54
СОДЕРЖАНИЕ	55
1. ВВЕДЕНИЕ.....	56
1.2. Краткая характеристика области применения программы.....	56
2. ОСНОВАНИЕ ДЛЯ РАЗРАБОТКИ	56
2.1. Основание для проведения разработки.....	56
2.2. Наименование и условное обозначение темы разработки.....	56
3. НАЗНАЧЕНИЕ РАЗРАБОТКИ.....	56
3.1. Функциональное назначение программы.....	56
3.2. Эксплуатационное назначение программы	56
4. ТРЕБОВАНИЯ К ПРОГРАММЕ.....	56
4.1. Требования к функциональным характеристикам	56
4.3. Условия эксплуатации	58
4.4. Требования к составу и параметрам технических средств.....	58
4.5. Требования к информационной и программной совместимости.....	59
5. ТРЕБОВАНИЯ К ПРОГРАММНОЙ ДОКУМЕНТАЦИИ	59
5.1. Предварительный состав программной документации	59
6. ТЕХНИКО-ЭКОНОМИЧЕСКИЕ ПОКАЗАТЕЛИ	59
6.2. Предполагаемая годовая потребность	59
7. СТАДИИ И ЭТАПЫ РАЗРАБОТКИ	60
7.1. Стадии разработки	60
7.2. Этапы разработки.....	60
7.3. Содержание работ по этапам	60
8. ПОРЯДОК КОНТРОЛЯ И ПРИЕМКИ.....	61

1. ВВЕДЕНИЕ

1.1. Наименование программы

Наименование - «Веб-приложение для бюро по трудоустройству».

1.2. Краткая характеристика области применения программы

Программа предназначена к применению в профильных подразделениях АСУ «НН», на автоматизируемых объектах ООО «НН».

2. ОСНОВАНИЕ ДЛЯ РАЗРАБОТКИ

2.1. Основание для проведения разработки

Основанием для проведения разработки является необходимость организации бюро по трудоустройству.

2.2. Наименование и условное обозначение темы разработки

Наименование темы разработки - «Разработка программы для бюро по трудоустройству».

Условное обозначение темы разработки (шифр темы) - «А.В.00001».

3. НАЗНАЧЕНИЕ РАЗРАБОТКИ

3.1. Функциональное назначение программы

Функциональным назначением программы является организация работы бюро по трудоустройству.

3.2. Эксплуатационное назначение программы

Программа должна эксплуатироваться в профильных подразделениях на объектах ООО «НН».

Конечными пользователями программы должны являться сотрудники профильных подразделений ООО «НН».

4. ТРЕБОВАНИЯ К ПРОГРАММЕ

4.1. Требования к функциональным характеристикам

4.1.1. Требования к составу выполняемых функций

Программа должна обеспечивать возможность выполнения перечисленных ниже функций:

- а) работа с данными фирмы;
- б) работа с данными пользователя;
- в) работа с данными пользователей системы;

г) работа с составлением договора по сделкам.

4.1.2. Требования к организации входных данных

Входные данные системы должны представляться в виде данных о пользователе, фирме.

Система должна обрабатывать данные о заключении договора по сделкам и о пользователе, фирме.

4.1.3. Требования к организации выходных данных

Выходные данные программы должны быть организованы в виде формирования договора о сделке. Отчеты делятся на несколько групп по предназначению определенной группе пользователей. Доступ к таблицам также зависит от принадлежности пользователя к определенной группе пользователя с теми или иными правами.

4.1.4. Требования к временным характеристикам

Требования к временным характеристикам программы не предъявляются.

4.2. Требования к надежности

4.2.1. Требования к обеспечению надежного (устойчивого) функционирования программы

Надежное (устойчивое) функционирование программы должно быть обеспечено выполнением совокупности организационно-технических мероприятий, перечень которых приведен ниже:

а) организацией бесперебойного питания технических средств;

б) регулярным выполнением рекомендаций Министерства труда и социального развития РФ, изложенных в Постановлении от 23 июля 1998 г. «Об утверждении межотраслевых типовых норм времени на работы по сервисному обслуживанию ПЭВМ и оргтехники и сопровождению программных средств»;

в) регулярным выполнением требований ГОСТ 51188-98. Защита информации. Испытания программных средств на наличие компьютерных вирусов;

г) необходимым уровнем квалификации сотрудников профильных подразделений.

4.2.2. Время восстановления после отказа

Время восстановления после отказа, вызванного сбоем электропитания технических средств (иными внешними факторами), не фатальным сбоем (не крахом) операционной системы, не должно превышать времени, необходимого на перезагрузку операционной системы и запуск программы, при условии соблюдения условий эксплуатации технических и программных средств.

Время восстановления после отказа, вызванного неисправностью технических средств, фатальным сбоем (крахом) операционной системы, не

должно превышать времени, требуемого на устранение неисправностей технических средств и переустановки программных средств.

4.2.3. Отказы из-за некорректных действий оператора

Отказы программы возможны вследствие некорректных действий оператора (пользователя) при взаимодействии с операционной системой. Во избежание возникновения отказов программы по указанной выше причине следует обеспечить работу конечного пользователя без предоставления ему административных привилегий.

4.3. Условия эксплуатации

4.3.1. Климатические условия эксплуатации

Климатические условия эксплуатации, при которых должны обеспечиваться заданные характеристики, должны удовлетворять требованиям, предъявляемым к техническим средствам в части условий их эксплуатации.

4.3.2. Требования к видам обслуживания

См. Требования к обеспечению надежного (устойчивого) функционирования программы.

4.3.3. Требования к численности и квалификации персонала

Минимальное количество персонала, требуемого для работы программы, должно составлять не менее 2 штатных единиц - системный программист и пользователь системы.

Системный программист должен иметь минимум среднее техническое образование. В перечень задач, выполняемых системным программистом, должны входить:

- а) задача поддержания работоспособности технических средств;
- б) задачи установки (инсталляции) и поддержания работоспособности системных программных средств - операционной системы;
- в) задача установки (инсталляции) программы.

Пользователь системы должен обладать практическими навыками работы с графическим пользовательским интерфейсом операционной системы.

Персонал должен быть аттестован минимум на II квалификационную группу по электробезопасности (для работы с конторским оборудованием).

4.4. Требования к составу и параметрам технических средств

В состав технических средств должен входить IBM-совместимый персональный компьютер (ПЭВМ), включающий в себя:

- а) процессор Pentium - 4 с тактовой частотой, 1.2 ГГц, не менее;
- б) оперативную память объемом, 128 Мб, не менее;
- в) жесткий диск объемом 40 Гб, и выше;
- г) оптический манипулятор типа «мышь»;
- д) наличие 2 COM-портов.

4.5. Требования к информационной и программной совместимости

4.5.1. Требования к информационным структурам и методам решения

Требования к информационным структурам на входе и выходе, а также к методам решения не предъявляются.

4.5.2. Требования к исходным кодам и языкам программирования

Исходные коды программы должны быть реализованы на языке С#. В качестве интегрированной среды разработки программы должна быть использована Microsoft Visual Studio 2022(локализованная, русская версия), Microsoft SQL Server Management Studio.

4.5.3. Требования к программным средствам, используемым программой

Системные программные средства, используемые программой, должны быть представлены локализованной версией операционной системы Windows 8, Windows 8.1, Windows 10.

4.5.4. Требования к защите информации и программ

Требования к защите информации и программ не предъявляются.

4.6. Специальные требования

Специальные требования к программе не предъявляются.

5. ТРЕБОВАНИЯ К ПРОГРАММНОЙ ДОКУМЕНТАЦИИ

5.1. Предварительный состав программной документации

Состав программной документации должен включать в себя:

- а) техническое задание;
- б) текст программы;
- в) описание программы;
- г) пояснительная записка;
- д) руководство администратора.

5.2. Специальные требования к программной документации

Специальные требования к программной документации не предъявляются.

6. ТЕХНИКО-ЭКОНОМИЧЕСКИЕ ПОКАЗАТЕЛИ

6.1. Ориентировочная экономическая эффективность

Ориентировочная экономическая эффективность не рассчитываются.

6.2. Предполагаемая годовая потребность

Предполагаемое число использования программы в год — круглосуточная работа программы на одном рабочем месте.

6.3. Экономические преимущества разработки

Экономические преимущества разработки не рассчитываются.

7. СТАДИИ И ЭТАПЫ РАЗРАБОТКИ

7.1. Стадии разработки

Разработка должна быть проведена в три стадии:

- а) разработка технического задания;
- б) рабочее проектирование;
- в) внедрение.

7.2. Этапы разработки

На стадии разработки технического задания должен быть выполнен этап разработки, согласования и утверждения настоящего технического задания.

На стадии рабочего проектирования должны быть выполнены перечисленные ниже этапы работ:

- а) разработка программы;
- б) разработка программной документации;
- в) испытания программы.

На стадии внедрения должен быть выполнен этап разработки - подготовка и передача программы.

7.3. Содержание работ по этапам

На этапе разработки технического задания должны быть выполнены перечисленные ниже работы:

- а) постановка задачи;
- б) определение и уточнение требований к техническим средствам;
- в) определение требований к программе;
- г) определение стадий, этапов и сроков разработки программы и документации на неё;
- д) выбор языков программирования;
- е) согласование и утверждение технического задания.

На этапе разработки программы должна быть выполнена работа по программированию и отладке программы.

На этапе разработки программной документации должна быть выполнена разработка программных документов в соответствии с требованиями ГОСТ 19.101-77.

«Предварительный состав программной документации» настоящего технического задания.

На этапе испытаний программы должны быть выполнены перечисленные ниже виды работ:

- а) разработка, согласование и утверждение программы и методики испытаний;

- б) проведение приемо-сдаточных испытаний;
- в) корректировка программы и программной документации по результатам испытаний.

На этапе подготовки и передачи программы должна быть выполнена работа по подготовке и передаче программы и программной документации в эксплуатацию.

8. ПОРЯДОК КОНТРОЛЯ И ПРИЕМКИ

Испытание представленной модели и контроль качества ее работы провести в учебной программе Teams в команде ЭИС-36_СУБД. Во время испытаний проверить работу системы по следующим позициям:

- а) Просмотр технического задания
- б) Проверка спроектированных моделей и анализ предметной области
- в) Запуск программы
- г) Проверка работоспособности функций программы
- д) Просмотр печатных форм
- е) Завершение показа программы

Приложение Б

SQL-запросы: создание и заполнение БД, триггер на уменьшение счёта компании при успешной сделке.

Создание БД:

```
create table должность_работника(  
должность_работника_id int primary key identity(1,1),  
название_должности_работника nvarchar(50) not null);  
create table адрес(  
адрес_id int primary key identity(1,1),  
город nvarchar(50) not null,  
улица nvarchar(50),  
дом nvarchar(10),  
квартира nvarchar(10));
```

```
create table роль(  
роль_id int primary key identity(1,1),  
название_роли nvarchar(50) not null);
```

```
create table паспорт(  
паспорт_id int primary key identity(1,1),  
серия nvarchar(4) not null,  
номер nvarchar(6) not null,  
дата_выдачи date not null,  
место_выдачи nvarchar(50) not null);
```

```
create table пользователь(  
пользователь_id int primary key identity(1,1),  
логин nvarchar(255) not null,  
пароль nvarchar(255) not null,  
роль_id int not null references роль(роль_id));
```

```
create table работник(  
работник_id int primary key identity(1,1),  
фамилия nvarchar(50) not null,  
имя nvarchar(50) not null,  
отчество nvarchar(50),  
должность_работника_id int not null references  
должность_работника(должность_работника_id),  
адрес_id int not null references адрес(адрес_id),  
паспорт_id int not null references паспорт(паспорт_id),  
пользователь_id int not null references пользователь(пользователь_id),  
телефон nvarchar(12) not null,  
фото nvarchar(255) not null,  
почта nvarchar(50) not null);
```

```

create table компания(
компания_id int primary key identity(1,1),
название_компании nvarchar(50) not null,
адрес_id int not null references адрес(адрес_id),
телефон nvarchar(12) not null,
почта nvarchar(50) not null,
информация_о_компании nvarchar(50) not null,
логотип_компании nvarchar(255) not null,
пользователь_id int not null references пользователь(пользователь_id),
счёт_компании nvarchar(10) not null);

```

```

create table образование(
образование_id int primary key identity(1,1),
уровень_образования nvarchar(50) not null,
направление_образования nvarchar(50) not null);

```

```

create table вид_деятельности(
вид_деятельности_id int primary key identity(1,1),
название_вида_деятельности nvarchar(50) not null)

```

```

create table вакансия(
вакансия_id int primary key identity(1,1),
название_вакансии nvarchar(50) not null,
вид_деятельности_id int not null references
вид_деятельности(вид_деятельности_id),
уровень_зп nvarchar(10) not null,
образование_id int not null references образование(образование_id),
компания_id int not null references компания(компания_id))

```

```

create table соискатель(
соискатель_id int primary key identity(1,1),
фамилия nvarchar(50) not null,
имя nvarchar(50) not null,
отчество nvarchar(50),
телефон nvarchar(12) not null,
почта nvarchar(50) not null,
информация nvarchar(100) not null,
фото nvarchar(255) not null,
вид_деятельности_id int not null references
вид_деятельности(вид_деятельности_id),
желаемая_зп nvarchar(10) not null,
образование_id int not null references образование(образование_id),
пользователь_id int not null references пользователь(пользователь_id))

```

```

create table сделка(

```

```
сделка_id int primary key identity(1,1),
работник_id int not null references работник(работник_id),
компания_id int not null references компания(компания_id),
соискатель_id int not null references соискатель(соискатель_id),
дата_сделки date not null,
стоимость nvarchar(10) not null,
вакансия_id int not null references вакансия(вакансия_id))
```

Заполнение БД:

```
insert into должность_работника(название_должности_работника) values
('Старший агент бюро');
```

```
insert into должность_работника(название_должности_работника) values
('Агент бюро');
```

```
insert into адрес(город, улица, дом, квартира) values ('Ярославль',
'Ньютона', 'NULL', 'NULL');
```

```
insert into адрес(город, улица, дом, квартира) values ('Ярославль',
'Углиская', 'NULL', 'NULL');
```

```
insert into адрес(город, улица, дом, квартира) values ('Ярославль',
'Московская', 'NULL', 'NULL');
```

```
insert into адрес(город, улица, дом, квартира) values ('Ярославль',
'Ульяновская', 'NULL', 'NULL');
```

```
insert into адрес(город, улица, дом, квартира) values ('Ярославль',
'Тагарина', 'NULL', 'NULL');
```

```
insert into адрес(город, улица, дом, квартира) values ('Ярославль',
'Углиская', 'NULL', 'NULL');
```

```
insert into адрес(город, улица, дом, квартира) values ('Ярославль',
'Савинова', 'NULL', 'NULL');
```

```
insert into роль(название_роли) values ('Администратор');
```

```
insert into роль(название_роли) values ('Работник');
```

```
insert into роль(название_роли) values ('Соискатель');
```

```
insert into роль(название_роли) values ('Компания');
```

```
insert into паспорт(серия, номер, дата_выдачи, место_выдачи) values
('2514','020304', '25.12.2000', 'УМВД по Ярославской области');
```

```
insert into паспорт(серия, номер, дата_выдачи, место_выдачи) values
('9585','030452', '20.10.2001', 'УМВД по Ярославской области');
```

```
insert into паспорт(серия, номер, дата_выдачи, место_выдачи) values
('7856','859652', '26.08.2007', 'УМВД по Ярославской области');
```

```
insert into паспорт(серия, номер, дата_выдачи, место_выдачи) values
('8596','758596', '03.12.2009', 'УМВД по Ярославской области');
```

```
insert into паспорт(серия, номер, дата_выдачи, место_выдачи) values
('5263','748596', '07.06.2010', 'УМВД по Ярославской области');
```



```

insert into пользователь(логин, пароль, роль_id) values ('smirnofff','1245',
1);
insert into пользователь(логин, пароль, роль_id) values
('Ivanov_ivan','5263', 2);
insert into пользователь(логин, пароль, роль_id) values ('rabotnik1','8596',
2);
insert into пользователь(логин, пароль, роль_id) values
('rabotnik_Lesha','4565', 2);
insert into пользователь(логин, пароль, роль_id) values
('rabotnik_Leon','8785', 2);
insert into пользователь(логин, пароль, роль_id) values
('kompany_stroi','8963', 4);
insert into пользователь(логин, пароль, роль_id) values
('kompany_IT','1245', 4);
insert into пользователь(логин, пароль, роль_id) values
('soiskatel_Andrei','7852', 3);
insert into пользователь(логин, пароль, роль_id) values
('soiskatel_Igor','9636', 3);

```

```

insert into работник(фамилия, имя, отчество, должность_работника_id,
адрес_id, паспорт_id, пользователь_id, телефон, фото, почта) values
('Смирнов','Иван', 'Иванович', 1, 1, 1, 1,'89962408711', '-', 'qwerty@gmail.com');
insert into работник(фамилия, имя, отчество, должность_работника_id,
адрес_id, паспорт_id, пользователь_id, телефон, фото, почта) values
('Иванов','Иван', 'Иванович', 2, 2, 2, 2,'89985523256', '-', 'ivanov@gmail.com');
insert into работник(фамилия, имя, отчество, должность_работника_id,
адрес_id, паспорт_id, пользователь_id, телефон, фото, почта) values
('Мандаринов','Михаил', 'Александрович', 2, 3, 3, 3,'8996896552', '-',
'mandarinov@gmail.com');
insert into работник(фамилия, имя, отчество, должность_работника_id,
адрес_id, паспорт_id, пользователь_id, телефон, фото, почта) values
('Чайников','Алексей', 'Иванович', 2, 4, 4, 4,'89968965241', '-',
'chainikov@gmail.com');
insert into работник(фамилия, имя, отчество, должность_работника_id,
адрес_id, паспорт_id, пользователь_id, телефон, фото, почта) values
('Мышкин','Леон', 'Алексеевич', 2, 5, 5, 5,'89968965263', '-',
'mishkin@gmail.com');

```

```

insert into компания(название_компании, адрес_id, телефон, почта,
информация_о_компании, логотип_компании, пользователь_id,
счёт_компании) values ('Компани-строй', 6, '899685296561',
'kompany_stroi@gmail.com', 'Молодая строительная компания', '-', 6, '500' );
insert into компания(название_компании, адрес_id, телефон, почта,
информация_о_компании, логотип_компании, пользователь_id,

```

```
счёт_компании) values ('Айти-компания', 7, '899685296561',  
'kompany_it@gmail.com', 'Молодая айти компания', '-', 7, '500' );
```

```
insert into образование(уровень_образования,  
направление_образования) values ('Высшее', 'техническое');
```

```
insert into образование(уровень_образования,  
направление_образования) values ('Высшее', 'строительное');
```

```
insert into образование(уровень_образования,  
направление_образования) values ('Среднее специальное', 'техническое');
```

```
insert into вид_деятельности(название_вида_деятельности) values  
('Строительство');
```

```
insert into вид_деятельности(название_вида_деятельности) values ('IT');
```

```
insert into вид_деятельности(название_вида_деятельности) values  
('Экономика');
```

```
insert into вид_деятельности(название_вида_деятельности) values  
('Управление');
```

```
insert into вакансия(название_вакансии, вид_деятельности_id,  
уровень_зп, образование_id, компания_id) values ('Молодой строитель-  
проектировщик', 1, 'от 30000', 2, 1);
```

```
insert into вакансия(название_вакансии, вид_деятельности_id,  
уровень_зп, образование_id, компания_id) values ('Опытный строитель  
многопрофильного уровня', 1, 'от 50000', 2, 1);
```

```
insert into вакансия(название_вакансии, вид_деятельности_id,  
уровень_зп, образование_id, компания_id) values ('Программист Java/C# с  
опытом', 2, 'от 100000', 1, 2);
```

```
insert into вакансия(название_вакансии, вид_деятельности_id,  
уровень_зп, образование_id, компания_id) values ('IT-аналитик в  
развивающуюся компанию', 2, 'от 120000', 1, 2);
```

```
insert into соискатель(фамилия, имя, отчество, телефон, почта,  
информация, фото, вид_деятельности_id, желаемая_зп, образование_id,  
пользователь_id) values ('Некелев', 'Андрей', 'Александрович', '85625623265',  
'nekelev@gmail.com', 'Строитель с интузиазмом', '-', 1, 'от 30000', 2, 8);
```

```
insert into соискатель(фамилия, имя, отчество, телефон, почта,  
информация, фото, вид_деятельности_id, желаемая_зп, образование_id,  
пользователь_id) values ('Наушников', 'Игорь', 'Алексеевич', '89962507845',  
'naushnikov@gmail.com', 'Опытный программист', '-', 2, 'от 100000', 1, 9);
```

Триггер на уменьшение счёта компании при успешной сделке:

```
USE [employment_agency]
```

```
GO
```

```
/****** Object: Trigger [dbo].[contact_trigger]    Script Date: 24.12.2022
```

```
7:54:59 *****/
```

```
SET ANSI_NULLS ON
```

```
GO
```

```
SET QUOTED_IDENTIFIER ON
```

```
GO
```

```
ALTER trigger [dbo].[contact_trigger]
```

```
on [dbo].[сделка]
```

```
after insert
```

```
as
```

```
    declare @price int
```

```
    set @price = (select стоимость from inserted)
```

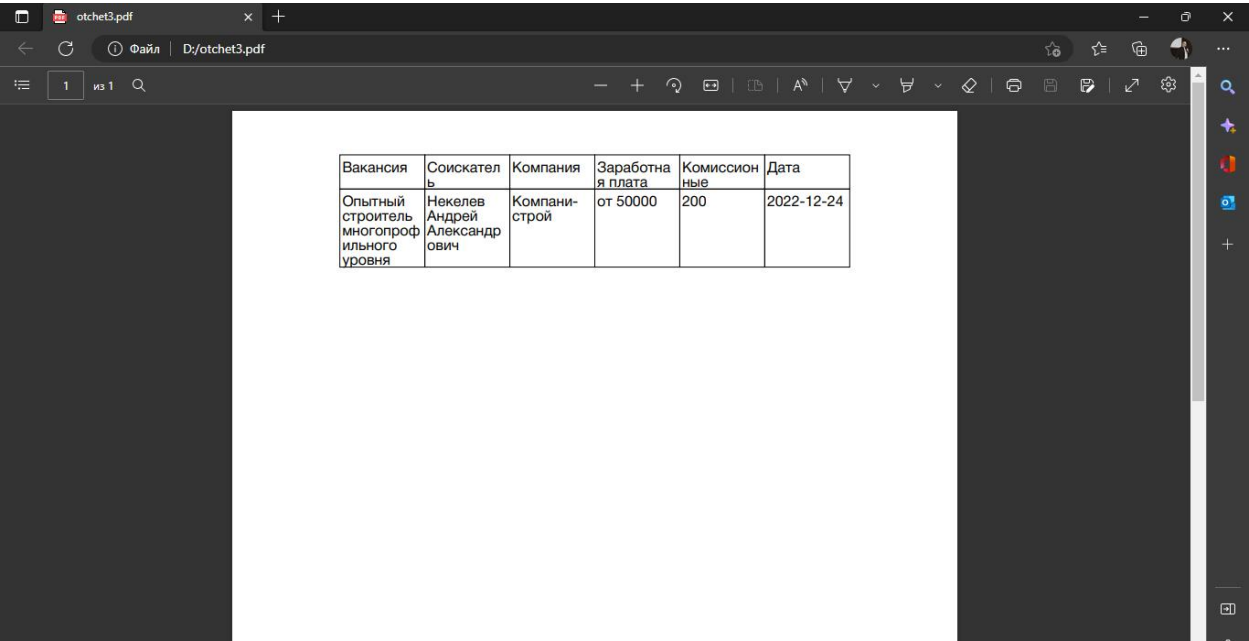
```
    update работодатель
```

```
    set счёт_компании = счёт_компании - @price
```

```
    where работодатель_id = (select работодатель_id from inserted)
```

Приложение В

Пример отчета



The image is a screenshot of a PDF viewer window. The title bar shows the file name 'otchet3.pdf'. The address bar shows the file path 'D:/otchet3.pdf'. The viewer interface includes a toolbar with various icons for navigation and editing. The main content area displays a table with the following data:

Вакансия	Соискатель	Компания	Заработная плата	Комиссионные	Дата
Опытный строитель многопрофильного уровня	Некелев Андрей Александрович	Компани-строй	от 50000	200	2022-12-24

Рисунок 44 – Пример отчёта .pdf

Приложение Г
Листинг наиболее важных частей кода

1) Авторизация

```
@WebServlet(urlPatterns = {"/LoginServlet"})
public class LoginServlet extends HttpServlet {
    ServletConfig conf;

    @Override
    public void init(ServletConfig config) {
        conf = config;
    }

    protected void processRequest(HttpServletRequest request,
        HttpServletResponse response)
        throws ServletException, IOException, SQLException,
        ClassNotFoundException, InstantiationException, IllegalAccessException {
        String us = request.getParameter("userName"); // логин
        String pass = request.getParameter("password"); // пароль

        // драйвер JDBC

        Class.forName("com.microsoft.sqlserver.jdbc.SQLServerDriver").newInstance();
        // подключение к БД
        String url =
        "jdbc:sqlserver://localhost:1433;databaseName=employment_agency";
        Connection con = DriverManager.getConnection(url, "drvyalitova", "0263");

        // SQL запрос
        Statement stmt = con.createStatement();
        String sql = "select * from пользователь "
            + "join роль on пользователь.роль_id = роль.роль_id "
            + "where логин = '"+us+"' and пароль = '"+pass+"'";

        // результат запроса
        ResultSet rs = stmt.executeQuery(sql);
        if (rs.next()) {
            // id и роль пользователя, который входит
            int id = rs.getInt("пользователь_id");
            String us_id = Integer.toString(id);
            String us_role = rs.getString("название_роли");
            // установка атрибутов
            ServletContext sc = conf.getServletContext();
            sc.setAttribute("us_id", us_id);
            sc.setAttribute("us_role", us_role);
            // вход
```

```

response.sendRedirect("http://localhost:8080/WebApplication1/profile.jsp");
    }
    else {
        response.sendRedirect("http://localhost:8080/WebApplication1/index.jsp");
    }
}

@Override
protected void doGet(HttpServletRequest request, HttpServletResponse
response)
    throws ServletException, IOException {
    try {
        processRequest(request, response);
    } catch (SQLException | ClassNotFoundException | InstantiationException |
IllegalAccessException ex) {
        Logger.getLogger(LoginServlet.class.getName()).log(Level.SEVERE, null,
ex);
    }
}

@Override
protected void doPost(HttpServletRequest request, HttpServletResponse
response)
    throws ServletException, IOException {
    try {
        processRequest(request, response);
    } catch (SQLException | ClassNotFoundException | InstantiationException |
IllegalAccessException ex) {
        Logger.getLogger(LoginServlet.class.getName()).log(Level.SEVERE, null,
ex);
    }
}
}

```

2) Вывод данных из БД

```

<%! String fam; %>
<%! String imya; %>
<%! String otc; %>
<%! String ser; %>
<%! String nom; %>
<%! Date pass_date; %>
<%! String kem; %>
<%! String phone; %>
<%! String email; %>

```

```

<%! String gor; %>
<%! String ul; %>
<%! String dom; %>
<%! String kvr; %>
<%! String foto; %>
<%! String dol; %>
<%! String vid_d; %>
<%! String level_o; %>
<%! String naprav; %>
<%! String money; %>
<%! String org_name; %>
<%! String info; %>
<%! int schet; %>
<%
// определение id и роли пользователя
    ServletContext sc = config.getServletContext();
    String us_id = (String)sc.getAttribute("us_id");
    String us_role = (String)sc.getAttribute("us_role");

// драйвер JDBC

Class.forName("com.microsoft.sqlserver.jdbc.SQLServerDriver").newInstance();
// подключение к БД
    String url =
"jdbc:sqlserver://localhost:1433;databaseName=employment_agency";
    Connection con = DriverManager.getConnection(url, "drvyalitova", "0263");

    Statement stmt = con.createStatement();
    String sql = null;
    if ("Работник бюро".equals(us_role) || "Администратор".equals(us_role)) {
        sql = "select * from пользователь "
            + "join работник on пользователь.пользователь_id =
работник.пользователь_id "
            + "join должность_работника on работник.должность_работника_id =
должность_работника.должность_работника_id "
            + "join адрес on работник.адрес_id = адрес.адрес_id "
            + "join паспорт on работник.паспорт_id = паспорт.паспорт_id "
            + "where пользователь.пользователь_id = "+us_id;

    ResultSet rs1 = stmt.executeQuery(sql);
    while (rs1.next()) {
// Работник
        fam = rs1.getString("фамилия");
        imya = rs1.getString("имя");
        otc = rs1.getString("отчество");
        phone = rs1.getString("телефон");

```

```

        email = rs1.getString("почта");
        foto = rs1.getString("фото");
        // Должность
        dol = rs1.getString("название_должности_работника");
        // Паспорт
        ser = rs1.getString("серия");
        nom = rs1.getString("номер");
        pass_date = rs1.getDate("дата_выдачи");
        kem = rs1.getString("место_выдачи");
        // Адрес
        gor = rs1.getString("город");
        ul = rs1.getString("улица");
        dom = rs1.getString("дом");
        kvr = rs1.getString("квартира");
    }
}

if ("Соискатель".equals(us_role)) {
    sql = "select * from пользователь "
        + "join соискатель on пользователь.пользователь_id =
соискатель.пользователь_id "
        + "join вид_деятельности on соискатель.вид_деятельности_id =
вид_деятельности.вид_деятельности_id "
        + "join образование on соискатель.образование_id =
образование.образование_id "
        + "where пользователь.пользователь_id = "+us_id;

    ResultSet rs2 = stmt.executeQuery(sql);
    while (rs2.next()) {
        // Соискатель
        fam = rs2.getString("фамилия");
        imya = rs2.getString("имя");
        otc = rs2.getString("отчество");
        phone = rs2.getString("телефон");
        email = rs2.getString("почта");
        info = rs2.getString("информация");
        foto = rs2.getString("фото");
        money = rs2.getString("желаемая_зп");
        // Вид деятельности
        vid_d = rs2.getString("название_вида_деятельности");
        // Образование
        level_o = rs2.getString("уровень_образования");
        naprav = rs2.getString("направление_образования");
    }
}

```



```

if ("Работодатель".equals(us_role)) {
    sql = "select * from пользователь "
        + "join работодатель on пользователь.пользователь_id =
работодатель.пользователь_id "
        + "join адрес on работодатель.адрес_id = адрес.адрес_id "
        + "where пользователь.пользователь_id = "+us_id;

```

```

ResultSet rs3 = stmt.executeQuery(sql);
while (rs3.next()) {
    // Работодатель
    org_name = rs3.getString("название_компании");
    info = rs3.getString("информация_o_компании");
    schet = rs3.getInt("счёт_компании");
    phone = rs3.getString("телефон");
    email = rs3.getString("почта");
    foto = rs3.getString("логотип_компании");

```

```

    // Адрес
    gor = rs3.getString("город");
    ul = rs3.getString("улица");
    dom = rs3.getString("дом");
    kvr = rs3.getString("квартира");
}
}

```

```

// необязательные

```

```

if (otc == null) {
    otc = "";
}

```

```

if (dom == null) {
    dom = "";
}
else {
    dom = " д."+dom;
}

```

```

if (kvr == null) {
    kvr = "";
}
else {
    kvr = " кв."+kvr;
}

```

```

%>

```

```

<!DOCTYPE html>
<html>

```

```

<head>
    <link rel="stylesheet" href="css/style.css">
    <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
    <title>Профиль</title>
</head>
<body>
    <center>
        <div class='side'>
            <form>
                <input type="submit" class="btn" value="Профиль" disabled><br><br>
                <%
                    if ("Работник бюро".equals(us_role) ||
"Администратор".equals(us_role)) {
                        out.println("<input type='submit' class='btn' name='contractButton'
value='Сделка'><br><br>");
                    }
                %>
                <input type='submit' class='btn' name='vacancyButton' value='Список
вакансий'><br><br>
                <input type='submit' class='btn' name='resumeButton' value='Список
резюме'><br><br>
                <%
                    if ("Администратор".equals(us_role)) {
                        out.println("<input type='submit' class='btn' name='usersButton'
value='Пользователи'><br><br>");
                    }
                %>
                <input type="submit" class="btn" name="exitButton" value="Выход">
            </form>
        </div>
        <%
            // переходы по меню
            if (request.getParameter("contractButton") != null) {

response.sendRedirect("http://localhost:8080/WebApplication1/contract.jsp");
            }
            if (request.getParameter("vacancyButton") != null) {

response.sendRedirect("http://localhost:8080/WebApplication1/vacancy.jsp");
            }
            if (request.getParameter("resumeButton") != null) {

response.sendRedirect("http://localhost:8080/WebApplication1/resume.jsp");
            }
            if (request.getParameter("usersButton") != null) {

```

```

response.sendRedirect("http://localhost:8080/WebApplication1/users.jsp");
    }
    if (request.getParameter("exitButton") != null) {

response.sendRedirect("http://localhost:8080/WebApplication1/index.jsp");
    }
    %>
    <div class='main'>
    %>
        if ("Соискатель".equals(us_role)) {
            out.println("<p>Ваше резюме</p>");
        }
        else {
            out.println("<p>Профиль</p>");
        }
    %>
    <table>
    <tr>
    <td>
        
    </td>
    <td>
        <%
            if ("Работник бюро".equals(us_role) ||
"Администратор".equals(us_role)) {
                out.println("<b>Роль:</b> "+us_role);
                out.println("<hr>");
                out.println("<b>ФИО:</b> "+fam+" "+imya+" "+otc+"<br>");
                out.println("<hr>");
                out.println("<b>Контактная информация:</b><br>");
                out.println("Телефон: "+phone+"<br>");
                out.println("E-mail: "+email);
                out.println("<hr>");
                out.println("<b>Адрес:</b><br>");
                out.println("Г. "+gor+" ул. "+ul + dom + kvr);
                out.println("<hr>");
                out.println("<b>Паспортные данные:</b><br>");
                out.println("Серия: "+ser+"<br>");
                out.println("Номер: "+nom+"<br>");
                out.println("Дата выдачи: "+pass_date+"<br>");
                out.println("Паспорт выдан "+kem);
            }
            if ("Соискатель".equals(us_role)) {
                out.println("<b>Роль:</b> "+us_role);
                out.println("<hr>");

```

```

        out.println("<b>ФИО:</b> "+fam+" "+imya+" "+otc);
        out.println("<hr>");
        out.println("<b>Контактная информация:</b><br>");
        out.println("Телефон: "+phone+"<br>");
        out.println("E-mail: "+email);
        out.println("<hr>");
        out.println("<b>О себе:</b><br>");
        out.println(info+"<br>");
        out.println("Вид деятельности: "+vid_d+"<br>");
        out.println("Желаемый уровень дохода: "+money);
        out.println("<hr>");
        out.println("<b>Образование:</b><br>");
        out.println("Уровень образования: "+vid_d+"<br>");
        out.println("Направление: "+naправ+"<br>");
    }
    if ("Работодатель".equals(us_role)) {
        out.println("<b>Роль:</b> "+us_role);
        out.println("<hr>");
        out.println("Название компании: "+org_name+"<br>");
        out.println("Счёт компании: "+schet);
        out.println("<hr>");
        out.println("<b>Контактная информация:</b><br>");
        out.println("Телефон: "+phone+"<br>");
        out.println("E-mail: "+email);
        out.println("<hr>");
        out.println("<b>Адрес:</b><br>");
        out.println("г. "+gor+" ул. "+ul + dom + kvr);
        out.println("<hr>");
        out.println("<b>О компании:</b><br>");
        out.println(info);
    }
    %>
</td>
</tr>
</table>
</div>
</center>
</body>
</html>

```

3) Добавление данных в БД

```

// вставка вакансии
if (request.getParameter("addVacancyButton") != null) {
    // параметры из формы

```

```

String vac_name = request.getParameter("vac_name"); vac_name = new
String(vac_name.getBytes("iso-8859-1"), "utf-8");
String vid_d = request.getParameter("vid_d"); vid_d = new
String(vid_d.getBytes("iso-8859-1"), "utf-8");
String ZP = request.getParameter("ZP"); ZP = new String(ZP.getBytes("iso-
8859-1"), "utf-8");
String level_o = request.getParameter("level_o"); level_o = new
String(level_o.getBytes("iso-8859-1"), "utf-8");
String naprav = request.getParameter("naprav"); naprav = new
String(naprav.getBytes("iso-8859-1"), "utf-8");

// SQL запросы
String sql1 = "insert into
вид_деятельности(название_вида_деятельности) "
+ "values('"+vid_d+"')";
stmt.executeUpdate(sql1);

int vid_d_id = 0;
String sql2 = "select max(вид_деятельности_id) from вид_деятельности";
ResultSet rs1 = stmt.executeQuery(sql2);
while (rs1.next()) {
    vid_d_id = rs1.getInt(1);
}

String sql3 = "insert into образование(уровень_образования,
направление_образования) "
+ "values('"+level_o+"', '"+naprav+"')";
stmt.executeUpdate(sql3);

int obr_id = 0;
String sql4 = "select max(образование_id) from образование";
ResultSet rs2 = stmt.executeQuery(sql4);
while (rs2.next()) {
    obr_id = rs2.getInt(1);
}

int rab_id = 0;
String sql5 = "select работодатель_id from работодатель "
+ "join пользователь on работодатель.пользователь_id =
пользователь.пользователь_id "
+ "where пользователь.пользователь_id = "+us_id;
ResultSet rs3 = stmt.executeQuery(sql5);
while (rs3.next()) {
    rab_id = rs3.getInt(1);
}

```

```
String sql6 = "insert into вакансии(название_вакансии,
вид_деятельности_id, уровень_зп, образование_id, работодатель_id) "
+ "values('"+vac_name+"', '"+vid_d_id+"', '"+ZP+"', '"+obr_id+",
"+rab_id+"')";
stmt.executeUpdate(sql6);
```

```
// переход
```

```
response.sendRedirect("http://localhost:8080/WebApplication1/vacancy.jsp");
}
```

```
// вставка сделки
```

```
if (request.getParameter("addContractButton") != null) {
    // параметры из формы
    String id_soisk = request.getParameter("id_soisk"); id_soisk = new
String(id_soisk.getBytes("iso-8859-1"), "utf-8");
    String id_vac = request.getParameter("id_vac"); id_vac = new
String(id_vac.getBytes("iso-8859-1"), "utf-8");
    String price = request.getParameter("price"); price = new
String(price.getBytes("iso-8859-1"), "utf-8");
```

```
// SQL запросы
```

```
int rab_id = 0;
String sql1 = "select работодатель.работодатель_id from вакансии "
+ "join работодатель on вакансии.работодатель_id =
работодатель.работодатель_id "
+ "where вакансии_id = '"+id_vac;
ResultSet rs1 = stmt.executeQuery(sql1);
while (rs1.next()) {
    rab_id = rs1.getInt(1);
}
```

```
Date d = new Date();
```

```
SimpleDateFormat formatForDate = new SimpleDateFormat("yyyy-MM-
dd");
```

```
String date = formatForDate.format(d);
```

```
String sql2 = "insert into сделка(работник_id, работодатель_id,
соискатель_id, дата_сделки, стоимость, вакансия_id) "
+ "values('"+us_id+", '"+rab_id+", '"+id_soisk+", '"+date+",
"+price+"', '"+id_vac+"')";
stmt.executeUpdate(sql2);
```

```
// переход
```

```
RequestDispatcher rd = request.getRequestDispatcher("/ReportServlet");
rd.forward(request, response);
```

```

    }

    // вставка соискателя
    if (request.getParameter("addSoiskButton") != null) {
        // параметры из формы
        String fam = request.getParameter("fam"); fam = new
String(fam.getBytes("iso-8859-1"), "utf-8");
        String imya = request.getParameter("imya"); imya = new
String(imya.getBytes("iso-8859-1"), "utf-8");
        String otc = request.getParameter("otc"); otc = new String(otc.getBytes("iso-
8859-1"), "utf-8");
        String phone = request.getParameter("phone"); phone = new
String(phone.getBytes("iso-8859-1"), "utf-8");
        String email = request.getParameter("email"); email = new
String(email.getBytes("iso-8859-1"), "utf-8");
        String info = request.getParameter("info"); info = new
String(info.getBytes("iso-8859-1"), "utf-8");
        String foto = request.getParameter("foto"); foto = new
String(foto.getBytes("iso-8859-1"), "utf-8");
        String vid_d = request.getParameter("vid_d"); vid_d = new
String(vid_d.getBytes("iso-8859-1"), "utf-8");
        String money = request.getParameter("money"); money = new
String(money.getBytes("iso-8859-1"), "utf-8");
        String level_o = request.getParameter("level_o"); level_o = new
String(level_o.getBytes("iso-8859-1"), "utf-8");
        String naprav = request.getParameter("naprav"); naprav = new
String(naprav.getBytes("iso-8859-1"), "utf-8");
        String login = request.getParameter("login"); login = new
String(login.getBytes("iso-8859-1"), "utf-8");
        String pass = request.getParameter("pass"); pass = new
String(pass.getBytes("iso-8859-1"), "utf-8");

        if ("".equals(otc)) {
            otc = null;
        }
        else {
            otc = ""+otc+"";
        }

        // SQL запросы
        String sql1 = "insert into пользователь(логин, пароль, роль_id) "
            + "values('"+login+"', '"+pass+"', 3)";
        stmt.executeUpdate(sql1);

        int id_user = 0;
        String sql2 = "select max(пользователь_id) from пользователь";
    }
}

```

```

ResultSet rs1 = stmt.executeQuery(sql2);
while (rs1.next()) {
    id_user = rs1.getInt(1);
}

String sql3 = "insert into образование(уровень_образования,
направление_образования) "
    + "values('"+level_o+"', '"+naправ+"')";
stmt.executeUpdate(sql3);

int obr_id = 0;
String sql4 = "select max(образование_id) from образование";
ResultSet rs2 = stmt.executeQuery(sql4);
while (rs2.next()) {
    obr_id = rs2.getInt(1);
}

String sql5 = "insert into
вид_деятельности(название_вида_деятельности) "
    + "values('"+vid_d+"')";
stmt.executeUpdate(sql5);

int vid_d_id = 0;
String sql6 = "select max(вид_деятельности_id) from вид_деятельности";
ResultSet rs3 = stmt.executeQuery(sql6);
while (rs3.next()) {
    vid_d_id = rs3.getInt(1);
}

String sql7 = "insert into соискатель(фамилия, имя, отчество, телефон,
почта, информация, фото, вид_деятельности_id, желаемая_зп, образование_id,
пользователь_id) "
    + "values('"+fam+"', '"+imya+"', '"+otc+"', '"+phone+"', '"+email+"',
'"+info+"', '"+foto+"', '"+vid_d_id+"', '"+money+"', '"+obr_id+"', '"+id_user+"')";
stmt.executeUpdate(sql7);

// переход
response.sendRedirect("http://localhost:8080/WebApplication1/index.jsp");
}

```

4) Редактирование данных

```

if (request.getParameter("updResumeButton") != null) {
    String id = request.getParameter("id");
    // параметры из формы

```



```

        String fam = request.getParameter("fam"); fam = new
String(fam.getBytes("iso-8859-1"), "utf-8");
        String imya = request.getParameter("imya"); imya = new
String(imya.getBytes("iso-8859-1"), "utf-8");
        String otc = request.getParameter("otc"); otc = new String(otc.getBytes("iso-
8859-1"), "utf-8");
        String phone = request.getParameter("phone"); phone = new
String(phone.getBytes("iso-8859-1"), "utf-8");
        String email = request.getParameter("email"); email = new
String(email.getBytes("iso-8859-1"), "utf-8");
        String info = request.getParameter("info"); info = new
String(info.getBytes("iso-8859-1"), "utf-8");
        String foto = request.getParameter("foto"); foto = new
String(foto.getBytes("iso-8859-1"), "utf-8");
        String vid_d = request.getParameter("vid_d"); vid_d = new
String(vid_d.getBytes("iso-8859-1"), "utf-8");
        String money = request.getParameter("money"); money = new
String(money.getBytes("iso-8859-1"), "utf-8");
        String level_o = request.getParameter("level_o"); level_o = new
String(level_o.getBytes("iso-8859-1"), "utf-8");
        String naprav = request.getParameter("naprav"); naprav = new
String(naprav.getBytes("iso-8859-1"), "utf-8");

```

```

int vid_d_id = 0;
int obr_id = 0;
String sql1 = "select * from соискатель where соискатель_id = "+id;
ResultSet rs1 = stmt.executeQuery(sql1);
while (rs1.next()) {
    vid_d_id = rs1.getInt("вид_деятельности_id");
    obr_id = rs1.getInt("образование_id");
}

```

```

String sql2 = "update вид_деятельности "
    + "set название_вида_деятельности = '"+vid_d+"' "
    + "where вид_деятельности_id = "+vid_d_id;
stmt.executeUpdate(sql2);

```

```

String sql3 = "update образование "
    + "set уровень_образования = '"+level_o+",
направление_образования = '"+naprav+"' "
    + "where образование_id = "+obr_id;
stmt.executeUpdate(sql3);

```

```

String sql4 = "update соискатель "

```

```
        + "set фамилия = '"+fam+"', имя = '"+imy+"', отчество = '"+otc+"',  
телефон = '"+phone+"', почта = '"+email+"', информация = '"+info+"', фото =  
 '"+foto+"', желаемая_зп = '"+money+" " "  
        + "where соискатель_id = "+id;  
stmt.executeUpdate(sql4);  
  
    // переход  
  
response.sendRedirect("http://localhost:8080/WebApplication1/resume.jsp");  
    }  
}
```

Приложение Д

Резервная копия БД

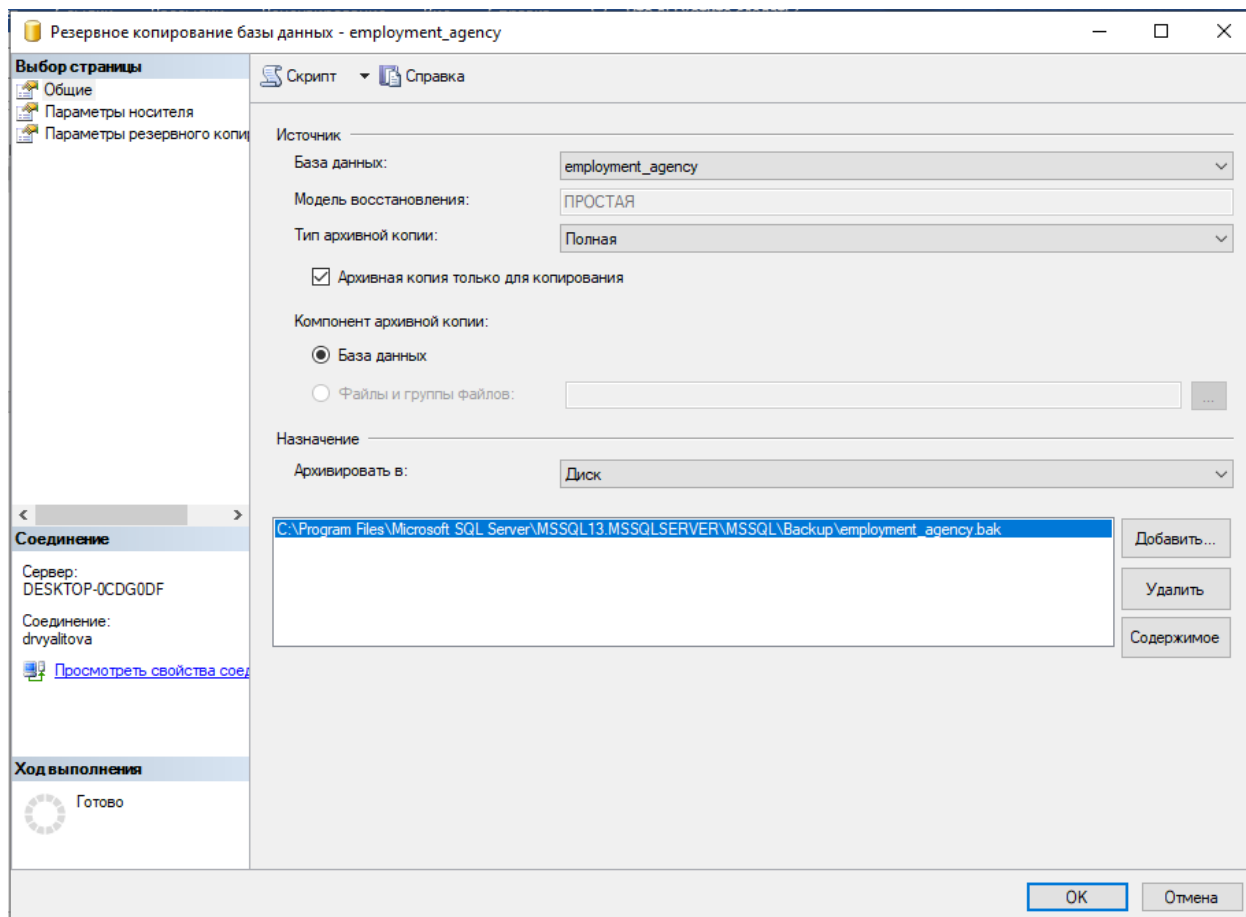


Рисунок 45 – Создание резервной копии

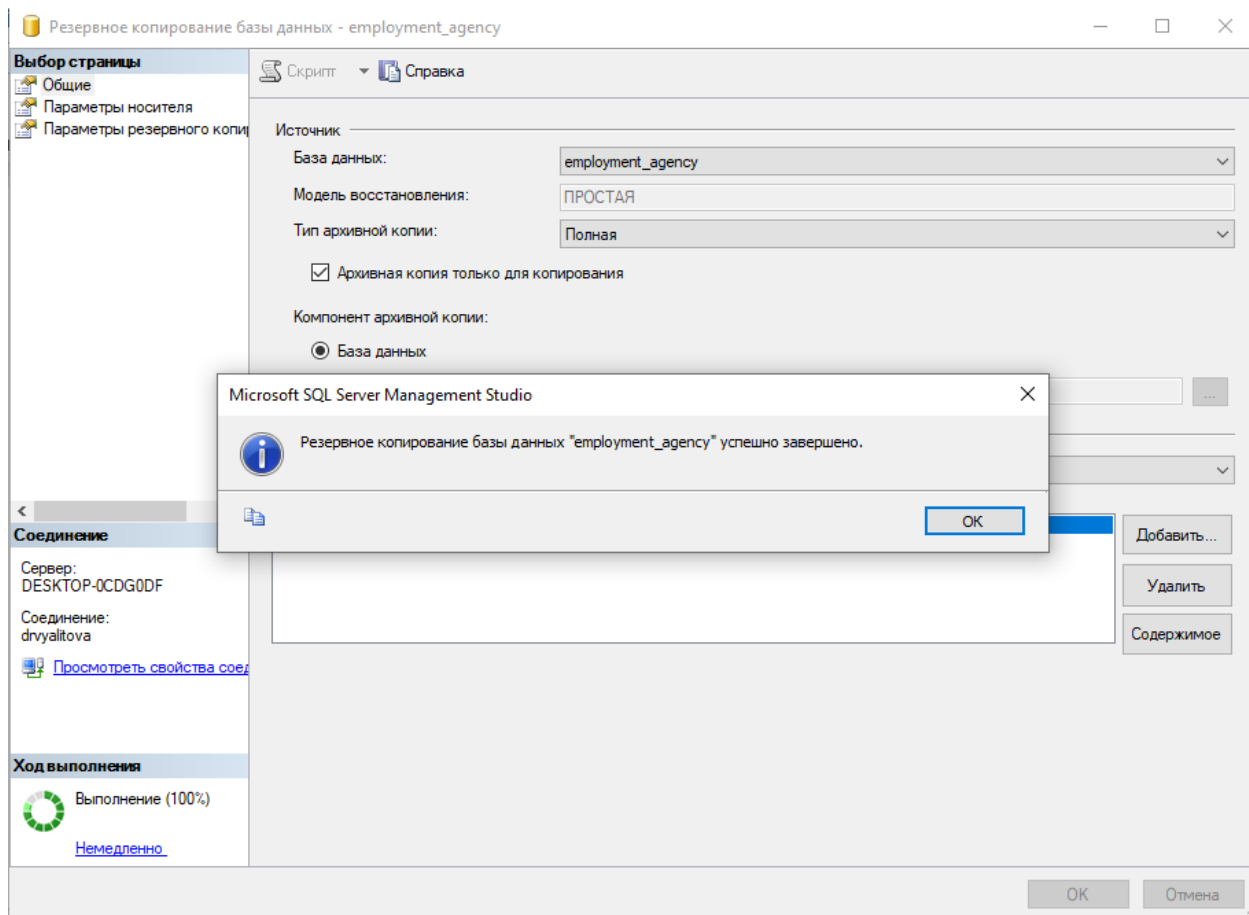


Рисунок 46 – Резервное копирование завершено